

Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης

Σχολή Θετικών Επιστημών

Τμήμα Πληροφορικής

Πτυχιακή Εργασία

Αυτορρύθμιση Τεχνικών Εύρεσης Ανωμαλιών

Author:
Ιωάννης Σταθόπουλος

Supervisor:
Prof. Αναστάσιος
Γούναρης



ΑΡΙΣΤΟΤΕΛΕΙΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΕΣΣΑΛΟΝΙΚΗΣ

15 Ιουνίου 2026

Δήλωση Πνευματικής Ιδιοκτησίας

Εγώ ο, Ιωάννης Σταθόπουλος, με πλήρη επίγνωση των συνεπειών του νόμου περί πνευματικών δικαιωμάτων, δηλώνω ότι η παρούσα πτυχιακή εργασία, καθώς και τα ηλεκτρονικά αρχεία και πηγαίοι κώδικες που αναπτύχθηκαν ή τροποποιήθηκαν στο πλαίσιο αυτής της εργασίας, αποτελεί αποκλειστικά προϊόν προσωπικής μου εργασίας, δεν προσβάλλει κάθε μορφής δικαιώματα διανοητικής ιδιοκτησίας, προσωπικότητας και προσωπικών δεδομένων τρίτων, δεν περιέχει έργα/εισφορές τρίτων για τα οποία απαιτείται άδεια των δημιουργών/δικαιούχων και δεν είναι προϊόν μερικής ή ολικής αντιγραφής, οι πηγές δε που χρησιμοποιήθηκαν περιορίζονται στις βιβλιογραφικές αναφορές και μόνον, και πληρούν τους κανόνες της επιστημονικής παράθεσης. Τα σημεία όπου έχω χρησιμοποιήσει ιδέες, κείμενο, αρχεία ή/και πηγές άλλων συγγραφέων, αναφέρονται ευδιάκριτα στο κείμενο με την κατάλληλη παραπομπή και η σχετική αναφορά περιλαμβάνεται στο τμήμα βιβλιογραφικών αναφορών με πλήρη περιγραφή. Αναλαμβάνω πλήρως, ατομικά και προσωπικά, όλες τις νομικές και διοικητικές συνέπειες που δύναται να προκύψουν στην περίπτωση κατά την οποία αποδειχθεί, διαχρονικά, ότι η εργασία αυτή ή τμήμα της δεν μου ανήκει, διότι είναι προϊόν λογοκλοπής.

ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ

Περίληψη

Τμήμα Πληροφορικής
Σχολή Θετικών Επιστημών

Προπτυχιακός Φοιτητής Πληροφορικής

Αυτορρύθμιση Τεχνικών Εύρεσης Ανωμαλιών

από Ιωάννης Σταθόπουλος

Η ανίχνευση ανωμαλιών σε χρονοσειρές, είναι ένα πολύπλευρο πρόβλημα, το οποίο έχει κεντρίσει το ενδιαφέρον τόσο των εταιριών όσο και των ακαδημαϊκών, καθώς σχετίζεται με υψηλότερης ποιότητας εφαρμογές στην υγεία, σε χρηματοοικονομικά συστήματα, σε περιβαλλοντολογικές παρακολούθησης καθώς και σε πολλούς άλλους τομείς.

Η ανάγκη για την καλύτερη ανίχνευση ανωμαλιών, προοικονομεί ότι πρέπει να υπάρχει ένα σύστημα, το οποίο να μπορεί να αποφασίζει ποιός αλγόριθμος ή ποιο σύνολο αλγορίθμων είναι καταλληλότερο ανάλογα με το είδος της χρονοσειράς, ένα τρόπος ώστε να υπολογίζονται αυτόματα οι διάφορες υπερπαραμέτροι του εκάστοτε αλγορίθμου, μία αμερόληπτη μετρική που να εφαρμόζεται σε όλα τα είδη των ανωμαλιών, καθώς επίσης και ένα σύστημα σημείου αναφοράς (benchmark), το οποίο θα αξιολογεί τις διάφορες τεχνικές ανίχνευσης ανωμαλιών που έχουν ή πρόκειται να προταθούν.

Στα πλαίσια αυτής της πτυχιακής εργασίας, θα μελετήσουμε το σύστημα AutoTSAD [1], ένα σύστημα το οποίο προτείνει μία αυτορυθμιζόμενη προσέγγιση για την ανίχνευση ανωμαλιών σε χρονοσειρές, θα συγκρίνουμε τα αποτελέσματά των πιο αποτελεσματικών μεμονωμένων αλγορίθμων ανίχνευσης ανωμαλιών, χρησιμοποιώντας την μετρική VUS-PR [2], πάνω σε δεδομένα τα οποία έχουν συλλεχτεί από τους δημιουργούς του TSB-UAD [3].

Ευχαριστίες

Ολοκληρώνοντας την πτυχιακή μου εργασία, θα ήθελα να ευχαριστήσω, αρχικά τον επιβλέποντα καθηγητή της εργασίας, κ. Αναστάσιο Γούναρη, προτίστως για την εμπιστοσύνη και την ευκαιρία που μου έδωσε να ασχοληθώ με το συγκεκριμένο θέμα, όπως επίσης και για την άμεση ανταπόκρισή και την συνεχή υποστήριξή του σε κάθε μου ερώτημα.

Επιπλέον ένα εγκάρδιο ευχαριστώ στην οικογένειά μου και κυρίως στην σύζυγό μου για την στήριξη που μου παρείχαν καθ' όλη τη διάρκεια αυτού του ταξιδιού, καθώς και να ζητήσω μία συγνώμη στις δύο μου κόρες, για όλες τις φορές που τους αρνήθηκα οποιαδήποτε δραστηριότητα μαζί τους, λόγω των ακαδημαϊκών υποχρεώσεων....

Περιεχόμενα

Δήλωση Πνευματικής Ιδιοκτησίας	iii
Περίληψη	v
Ευχαριστίες	vii
1 Εισαγωγή	1
1.1 Το Πρόβλημα	1
1.2 Προτεινόμενες Λύσεις	1
1.2.1 TSB-UAD Σύστημα σημείου αναφοράς (benchmark) για την ανίχνευση ανωμαλιών σε χρονοσειρές	2
1.2.2 AutoTSAD: Ένα αυτοματοποιημένο σύστημα για την ανίχνευση ανωμαλιών σε χρονοσειρές	2
1.3 Σκοπός Πτυχιακής	2
2 Θεωρητικό Υπόβαθρο	3
2.1 Χρονοσειρές	3
2.2 Ανωμαλίες	3
2.3 Μεμονωμένοι Αλγόριθμοι Σύγκρισης	6
2.3.1 Isolation Forest	6
2.3.2 NormA	9
2.3.3 Matrix Profile - MP	12
2.3.4 POLY	16
2.3.5 AutoTSAD: Unsupervised Holistic Anomaly Detection for Time Series Data	17
Εισαγωγή και Πλαισίωση του Προβλήματος	17
Αρχιτεκτονική Συστήματος	18
Θεμελιώδης Καινοτομίες και Πλεονεκτήματα του Συστήματος	18
Αξιολόγηση Απόδοσης και Ποιότητα Ανίχνευσης	20
Συμπεράσματα	20
3 Πειραματική Διάταξη και Συγκριτική αξιολόγηση	23
3.1 Γενικά	23
3.1.1 Μετρικές Αξιολόγησης	23
3.1.2 Γενικές Παρατηρήσεις	24
3.1.3 TODO	24
3.2 Public Dataset	24
3.2.1 Τα dataset	25
3.2.2 Χρόνοι εκτέλεσης	27
3.2.3 Αποτελέσματα ROC-VUS	31
Συγκριτικά αποτελέσματα με Optuna-ON	32
Συγκριτικά αποτελέσματα με Optuna-OFF	35

3.3 Artificial Dataset	36
3.4 ΚΕΡΔΟΣ ΑΝΑ ΧΡΟΝΟ	36
A' Frequently Asked Questions	39
A'.1 How do I change the colors of links?	39
Βιβλιογραφία	41

Κατάλογος σχημάτων

2.1 Τύποι ανωμαλιών σε χρονοσειρές (αναπαραγωγή από [3]) . .	4
2.2 Amplitude anomaly (αναπαραγωγή από [19])	4
2.3 Extremum anomaly (αναπαραγωγή από [19])	5
2.4 Frequency anomaly (αναπαραγωγή από [19])	5
2.5 Mean anomaly (αναπαραγωγή από [19])	6
2.6 Pattern-sine anomaly (αναπαραγωγή από [19])	6
2.7 Pattern-ecg anomaly (αναπαραγωγή από [19])	7
2.8 Pattern-shift anomaly (αναπαραγωγή από [19])	7
2.9 Platform anomaly (αναπαραγωγή από [19])	8
2.10 Trend anomaly (αναπαραγωγή από [19])	8
2.11 Variance anomaly (αναπαραγωγή από [19])	9
2.12 Mode correlation anomaly (αναπαραγωγή από [19])	9
2.13 Χρονοσειρές με διαφορετικούς τύπους ανωμαλιών (αναπαραγωγή από [3])	10
2.14 (a) Απομόνωση κανονικού σημείου x_i (b) Απομόνωση ανώμαλου σημείου x_0 (c) Μέση απόσταση σημείων x_0 και x_i από ρίζα δένδρου, με αυξανόμενο αριθμό δένδρων (αναπαραγωγή από [5])	11
2.15 Ανίχνευση ανωμαλιών με τον <i>Matrix Profile</i> (αναπαραγωγή από [15])	13
2.16 (αναπαραγωγή από [15])	15
2.17 (αναπαραγωγή από [15])	15
2.18 Επισκόπηση της αρχιτεκτονικής του AutoTSAD	19
2.19 (αριστερά) Κατανομή χρόνου εκτέλεσης του AutoTSAD με βελτιστοποίηση (O+) και χωρίς βελτιστοποίηση (O-) για τα dataset (A) TSB-UAD-artificial-69_2_0.02_15, (B) KDD-TSAD-022, and (C) SAND-SED, (δεξιά) διάμεσοι χρόνοι εκτέλεσης για όλα τα dataset χωρίς βελτιστοποίηση για διαφορετικούς αριθμούς παραλληλισμού	20
2.20 Ποιότητα Ανίχνευσης Ανωμαλιών (RANGE-PR-AUC) του <i>AutoTSAD</i> συγκριτικά με άλλους baseline αλγόριθμους-συστήματα, σε όλα τα επιτυχώς επεξεργασμένα datasets	21
3.1 Ποσοστά επιτυχίας ανίχνευσης τουλάχιστον μίας ανωμαλίας του Auto-TSAD	25
3.2 Χρόνοι εκτέλεσης για κάθε dataset με ανοιχτή την μηχανή βελτιστοποίησης optuna, σε ώρες.	28
3.3 Κώδικας για την εκτέλεση του IForest μέσω του TSB-UAD . .	30
3.4 Κώδικας για την εκτέλεση του POLY μέσω του TSB-UAD . . .	30
3.5 Κώδικας για την εκτέλεση του MP μέσω του TSB-UAD	31

3.6	Κώδικας για την εκτέλεση του MP μέσω του TSB-UAD. Πάνω αριστερά διακρίνουμε τις χρονικές απαιτήσεις του IForest. Πάνω δεξιά της χρονικές απαιτήσεις του POLY. Κάτω αριστερά διακρίνουμε τις χρονικές απαιτήσεις του MP. Κάτω δεξιά διακρίνουμε συγκριτικά τις χρονικές απαιτήσεις όλων των αλγόριθμων.	31
3.7	Συγκριτικά αποτελέσματα αλγορίθμων IForest - POLY - MP στην μετρική ROC-VUS.	33
3.8	Αποτελέσματα χρήσης Optuna σε φθίνουσα σειρά.	35
3.9	Σύγκριση μεταξύ όλων των τεχνικών.	36
3.10	Σύγκριση μεταξύ όλων των τεχνικών.	37
3.11	Αποτελέσματα Auto-TSAD με Optuna-OFF στο v2_public dataset.	37
3.12	Σύγκριση αποτελεσμάτων μεταξύ Auto-TSAD χωρίς Optuna και TSB-UAD.	37

Κατάλογος πινάκων

3.1 Experiments For Auto-TSAD with/out for each dataset	23
3.2 Στατιστικά χρόνου εκτέλεσης με την optuna στο publicv2 dataset	28
3.3 Χρόνοι εκτέλεσης σε sec στο dataset public v2, χωρίς ενεργή την μηχανή βελτιστοποίησης	29
3.4 Χρόνοι εκτέλεσης σε sec στο dataset public v2, χωρίς ενεργή την μηχανή βελτιστοποίησης και χωρίς ακραίες τιμές	30
3.5 Νικητήριοι Αλγόριθμοι ανά Dataset	34

For/Dedicated to/To my...

Κεφάλαιο 1

Εισαγωγή

1.1 Το Πρόβλημα

Η είσοδος στην καθημερινότητα μας, των συσκευών που συνδέονται στο Διαδύκτιο (IoT), έχει επηρεάσει αισθητά των όγκο των δεδομένων που συλλέγονται καθημερινά, καθώς και την ποικιλία τους. Το μεγαλύτερο ποσοστό αυτών των δεδομένων, είναι στην μορφή χρονοσειράς, δηλαδή μία ακολουθία από τιμές που έχουν καταγραφεί σε διαδοχικά χρονικά διαστήματα. Η ανίχνευση ανωμαλιών στις χρονοσειρές, είναι ένα πολύπλευρο πρόβλημα, το οποίο έχει κεντρίσει το ενδιαφέρον, τόσο των εταιριών αλλά και τον επιστημονικό κόσμο, καθώς σχετίζεται με υψηλότερης ποιότητας εφαρμογές στην υγεία, σε χρηματοοικονομικά συστήματα, σε περιβαλλοντολογικές παρακολουθήσεις, καθώς και σε πολλούς άλλους τομείς. Από την ανίχνευση ανωμαλιών μπορούμε να εξάγουμε πολύτιμες πληροφορίες, όπως π.χ. την ανίχνευση μία απάτης σε μία πιστωτική κάρτα, ή την ανίχνευση μίας σεισμικής δραστηριότητας, τον εντοπισμό ασυνήθιστης συμπεριφοράς ενός χρήστη σε ένα σύστημα κ.ο.κ.

Τις τελευταίες δεκαετίες, έχουν προταθεί και αξιολογηθεί πληθώρα αλγορίθμων ανίχνευσης ανωμαλιών σε χρονοσειρές [3]–[16]. Ωστόσο η επιλογή του κατάλληλου αλγορίθμου και η ρύθμιση των υπερπαραμέτρων του, είναι ένα πολυσύνθετο πρόβλημα, το οποίο εξαρτάται από το είδος της χρονοσειράς, των τύπο των ανωμαλιών, και απαιτεί πολύ χρόνο και υπολογιστικό κόστος.

Επίσης η έλλειψη ύπαρξης δεδομένων με ετικέτες για την εκπαίδευση των αλγορίθμων, μας οδηγεί σε μία μη-επιβλεπόμενη τεχνική για την ανίχνευση ανωμαλιών.

Τέλος, η έλλειψη ύπαρξης ενός καθολικού συστήματος σημείου αναφοράς (benchmark), το οποίο θα αξιολογεί τις διάφορες τεχνικές ανίχνευσης ανωμαλιών που έχουν ή πρόκειται να προταθούν, καθώς και μίας αμερόληπτης τεχνικής πέραν των παραδοσιακών, συνθέτουν ακόμα δύο προβλήματα προς εξερεύνηση.

1.2 Προτεινόμενες Λύσεις

Τα τελευταία χρόνια (από το και μετά), έχουν προταθεί κάποια συστήματα, τα οποία έρχονται στοχεύουν στην επίλυση των προαναφερθέντων προβλημάτων.

1.2.1 TSB-UAD Σύστημα σημείου αναφοράς (benchmark) για την ανίχνευση ανωμαλιών σε χρονοσειρές

Το 2022, προτάθηκε το «TSB: A Time Series Benchmark for Anomaly Detection» [3], το οποίο αποτελείται από από μία συλλογή αποτελούμενη από 13766 χρονοσειρές εκ των οποίων, οι 1980 προέρχονται από πραγματικά δεδομένα, οι 958 από τεχνητά δεδομένα, και 10828 συνθετικές χρονοσειρές κλιμακούμενης δυσκολίας.

Είναι ένα end-to-end benchmark, για μονοδιάστατες χρονοσειρές, το οποίο περιλαμβάνει μια ευρεία γκάμα τύπων ανωμαλιών. Προσπαθεί να παρέχει μία εννοιαία πλατφόρμα, σύγκρισης μεθόδων σε διαφορετικά ρεαλιστικά σενάρια, και να παρέχει ένα σύνολο από εργαλεία για την αξιολόγηση των αλγορίθμων ανίχνευσης ανωμαλιών σε χρονοσειρές.

1.2.2 AutoTSAD: Ένα αυτοματοποιημένο σύστημα για την ανίχνευση ανωμαλιών σε χρονοσειρές

Το 2024, προτάθηκε το «AutoTSAD: Unsupervised Holistic Anomaly Detection for Time Series Data» [1] το οποίο είναι ένα μη-επιβλεπόμενο συνδιαστικό σύστημα ανίχνευσης ανωμαλιών σε μονοδιάστατες χρονοσειρές. Βασικά χαρακτηριστικά του είναι ότι δεν απαιτεί την ύπαρξη ετικετών για την αξιολόγηση των αλγορίθμων, βασίζεται στον μηχανισμό Optuna [17] για την εύρεση των βέλτιστων υπερπαραμέτρων, και χρησιμοποιεί ένα ευρύ φάσμα αλγορίθμων ανίχνευσης ανωμαλιών. Επίσης σε αντίθεση με άλλα συστήματα, το AutoTSAD επιστρέφει μία συνάθροιση βαθμολογιών, και όχι μία κατάταξη των αλγορίθμων.

Τέλος μπορεί να λειτουργήσει και χωρίς την χρήση του μηχανισμού Optuna, παρά μόνο με προκαθορισμένες τιμές για τις υπερπαραμέτρους του κάθε αλγορίθμου, λόγω του τεράστιου απαιτούμενου χρόνου για την εύρεση των βέλτιστων υπερπαραμέτρων.

1.3 Σκοπός Πτυχιακής

Σκοπός αυτής της πτυχιακής εργασίας, είναι η αξιολόγηση του AutoTSAD, το οποίο είναι ένα χρονικά απαιτητικό σύστημα, και να συγκριθεί η επίδοσή του, με μεμονωμένους αλγόριθμους, οι οποίοι έχουν καλή απόδοση ο καθένας σε διαφορετικά είδη τύπου ανωμαλιών, όπως ο MP [14], ο IForest [5], ο POLY [18] και NORMA **mahin2024norma**, να παρουσιαστούν τα αποτελέσματα, και να αξιολογηθεί αν υπάρχει σοβαρή διαφορά η οποία να δικαιολογεί το χρονικό κόστος, πάντα με την χρήση αμερόληπτων τεχνικών, όπως η VUS-ROC[2].

Κεφάλαιο 2

Θεωρητικό Υπόβαθρο

Σε αυτό το κεφάλαιο, θα δώσουμε κάποιους ορισμούς, για έννοιες που θα παρουσιαστούν αργότερα στα πλαίσια της εργασίας.

2.1 Χρονοσειρές

Χρονοσειρά ορίζεται ως μία ακολουθία από παρατηρήσεις, οι οποίες συλλέγονται ανά κανονικά/ακανόνιστα χρονικά διαστήματα, είναι διατεταγμένες σε χρονική σειρά, και λαμβάνονται από ιατρικά, οικονομικά, IoT, αισθητήρες, κ.ο.κ. Μπορεί να είναι μονοδιάστατες ή πολυδιάστατες, ανάλογα με τον αριθμό των μεταβλητών της κάθε μέτρησης. Στα πλαίσια της εργασίας θα ασχοληθούμε μόνο με μονοδιάστατες χρονοσειρές.

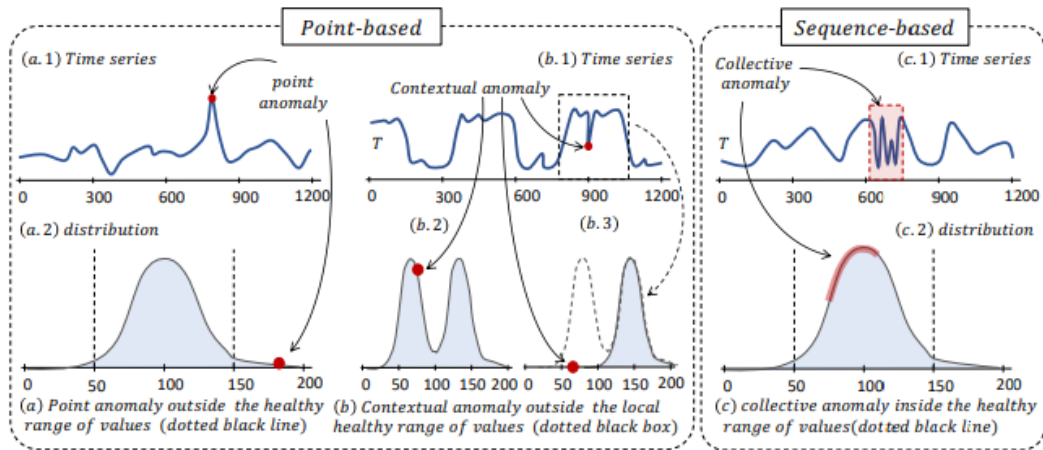
Συγκεκριμένα σύμφωνα με το [1, Section 3.1], ως μονοδιάστατες χρονοσειρές $T \in \mathbb{R}^n$ ορίζεται μία ακολουθία αληθινών μετρήσεων $T_i \in \mathbb{R}$, όπου $i \in |T|$ και $|T| = n$. Το μήκος της χρονοσειράς T είναι $|T|$ ή n και το i -οστό σημείο της ακολουθίας συμβολίζεται ως T_i .

2.2 Ανωμαλίες

Ανωμαλίες σε χρονοσειρές σύμφωνα με το [3, Section 3.1], ορίζονται ως σημεία, ή ακολουθίες σημείων, όπου αποκλίνουν σημαντικά από την κανονική συμπεριφορά της ακολουθίας, ή μίας υπακολουθίας της. Μπορεί να οφείλονται σε εσφαλμένες μετρήσεις ή σε πραγματικά ασυνήθιστα γεγονότα. Χωρίζονται σε τρεις κατηγορίες:

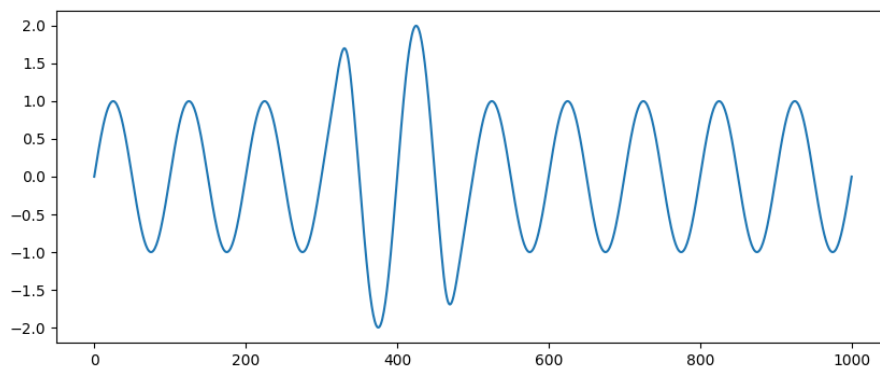
- **Point Anomalies:** Αφορούν μεμονωμένα σημεία τα οποία διαφέρουν σημαντικά από τα υπόλοιπα σημεία της χρονοσειράς.
- **Contextual Anomalies:** Αφορούν σημεία, τα οποία μπορεί να είναι φυσιολογικά σε σχέση με τα υπόλοιπα σημεία της χρονοσειράς, αλλά να διαφέρουν σημαντικά δεδομένου ενός συγκεκριμένου πλαισίου.
- **Collective anomalies:** Αφορούν ακολουθίες σημείων, τα οποία δεν επαναλαμβάνουν ένα κανονικό, παρατηρήσιμο μοτίβο. Αξίζει να σημειωθεί, πως αν τα σημεία της υπακολουθίας παρατηρηθούν ως μεμονωμένα, μπορεί να ανιχνευτούν ως κανονικά από έναν αλγόριθμο ανίχνευσης ανωμαλιών.

Οι δημιουργοί του *GutenTAG*[19] (βιβλιοθήκη για δημιουργία τεχνητών χρονοσειρών με ή χωρίς ανωμαλίες), τις κατατάσσουν στις εξής κατηγορίες:



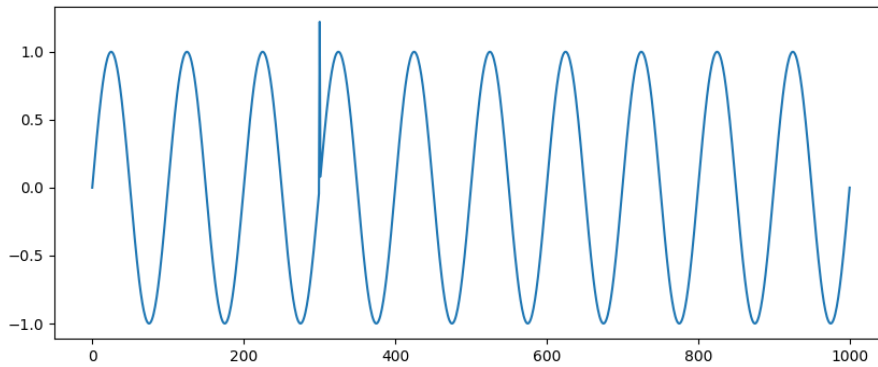
Σχήμα 2.1: Τύποι ανωμαλιών σε χρονοσειρές (αναπαραγωγή από [3])

- **Amplitude:** Αφορά στην αλλαγή του πλάτους του σήματος για κάποια χρονική περίοδο (Σχήμα 2.2).

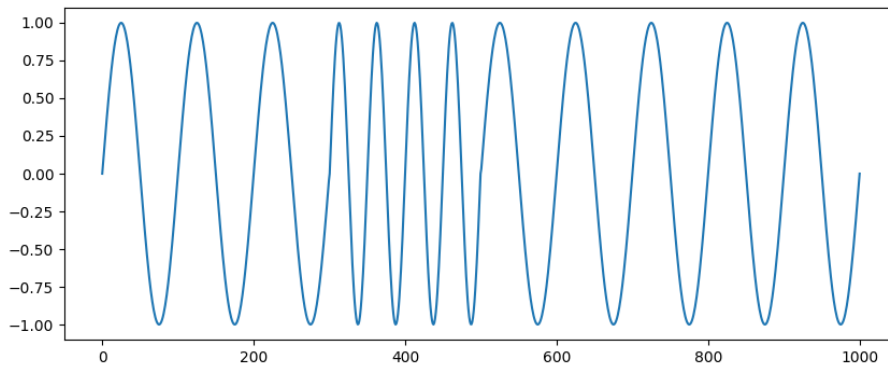


Σχήμα 2.2: Amplitude anomaly (αναπαραγωγή από [19])

- **Extremum:** Αφορά την εμφάνιση μίας ακραία τιμής (spike) στο σήμα, το οποίο μπορεί να είναι είτε τοπικά μέγιστο-ελάχιστο, ή καθολικά μέγιστο-ελάχιστο (Σχήμα 2.3).
- **Συχνότητας:** Αφορά στην αλλαγή της συχνότητας σε μία χρονική περίοδο ενός σήματος (Σχήμα 2.4).
- **Mean:** Αφορά την κατακόρυφη μετατόπιση της χρονοσειράς κατά μία σταθερή τιμή για το διάστημα της ανωμαλίας (Σχήμα 2.5).
- **Pattern:** Αφορά στην αλλαγή του επαναλαμβανόμενου μοτίβου της χρονοσειράς (π.χ. ένα ημιτονοειδές σήμα θα μπορούσε να παραμορφωθεί για το διάστημα της ανωμαλίας) (Σχήμα 2.7 και 2.8).
- **Pattern Shift:** Αφορά την χρονική μετατόπιση του μοτίβου της χρονοσειράς (Σχήμα 2.8).



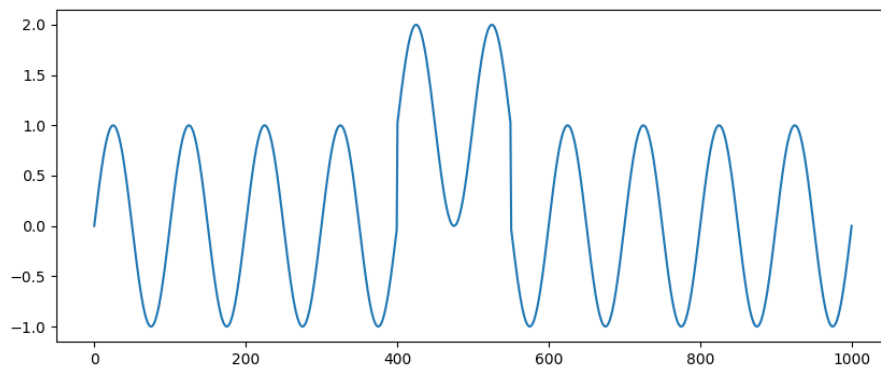
Σχήμα 2.3: Extremum anomaly (αναπαράγωγή από [19])



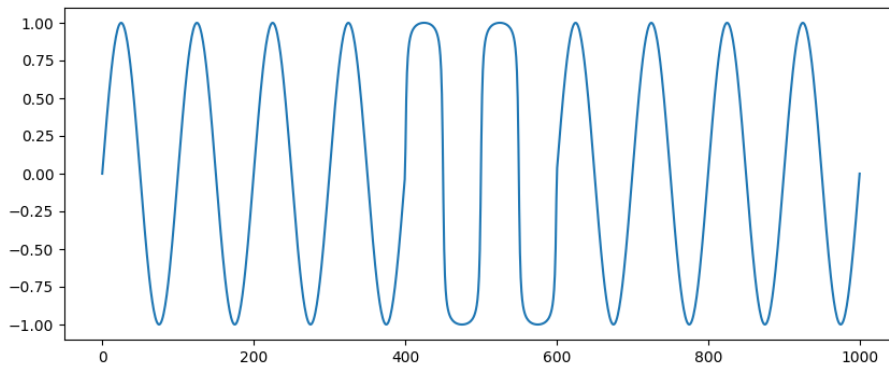
Σχήμα 2.4: Frequency anomaly (αναπαράγωγή από [19])

- **Platform:** Για κάποιο χρονικά διάστημα, η χρονοσειρά "παγώνει" και έπειτα συνεχίζει κανονικά (π.χ. θα μπορούσαμε να το προσομοιάσουμε με την στιγμή που η καρδιά ενός ασθενή σταματάει να δουλεύει για κάποια δευτερολέπτα, ο ασθενής επαναφέρεται και η καρδιά δουλεύει πάλι κανονικά) (Σχήμα 2.9).
- **Trend:** Αφορά σε μία αυξανόμενη ή μειούμενη τάση στην χρονοσειρά (Σχήμα 2.10).
- **Variance:** Αφορά στην μεταβολή του επίπεδου θορύβου για μία χρονική περίοδο της χρονοσειράς (Σχήμα 2.11).
- **Mode Correlation:** Αφορά στην αλλαγή της συμπεριφοράς μίας υποκολουθίας για το διάστημα που κρατάει η ανωμαλία (Σχήμα 2.12).

Είναι πολύ πιθανό, σε χρονοσειρές πραγματικών δεδομένων, να παρατηρηθούν παραπάνω από ένας τύπος ανωμαλιών, ή ακόμα και συνδυασμοί αυτών. Τέτοια παραδείγματα φαίνονται στο 2.13. Στην (d.1) για παράδειγμα, μπορούμε να παρατηρήσουμε ένα *point anomaly*, το A2 και ένα *collective anomaly*, την υποκολουθία A1.



Σχήμα 2.5: Mean anomaly (αναπαραγωγή από [19])



Σχήμα 2.6: Pattern-sine anomaly (αναπαραγωγή από [19])

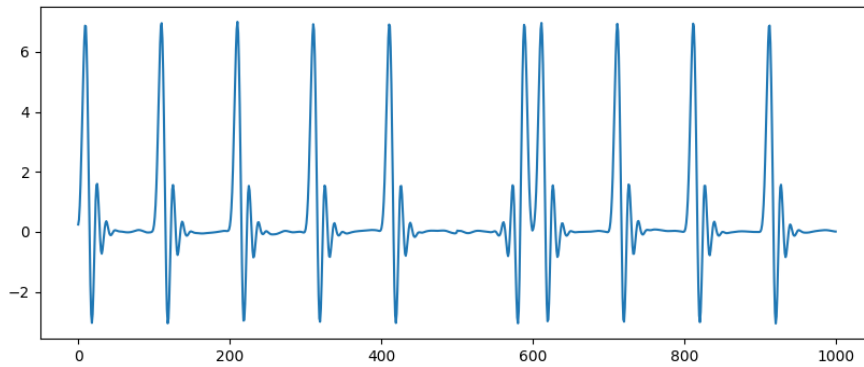
2.3 Μεμονωμένοι Αλγόριθμοι Σύγκρισης

Σε αυτό το σημείο, θα κάνουμε μία σύντομη αναφορά στους αλγορίθμους που αναφέρθηκαν στο 1.3 και θα χρησιμοποιηθούν στην εργασία. Θα αναφερθεί ο τρόπος λειτουργίας τους, τα πλεονεκτήματα και τα μειονεκτήματά τους, πώς συνδέονται με την ανίχνευση ανωμαλιών, και πώς οι υπερπαράμετροί τους επηρεάζουν την απόδοσή τους.

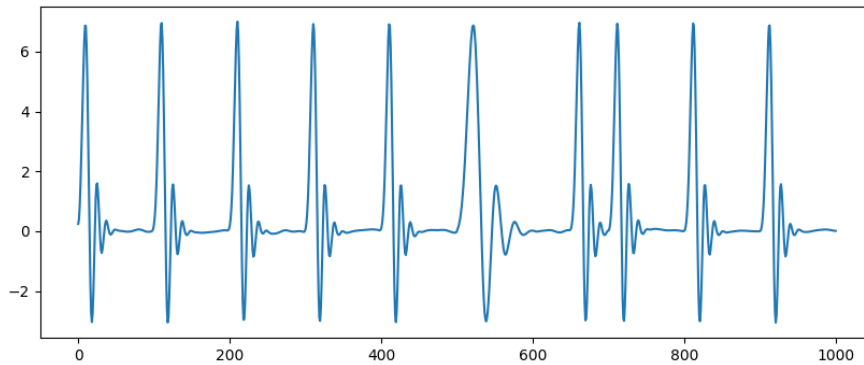
2.3.1 Isolation Forest

Οι περισσότεροι αλγόριθμοι ανίχνευσης ανωμαλιών, δημιουργούν ένα μοτίβο κανονικών δεδομένων και έπειτα, τα σημεία τα οποία αποκλίνουν από αυτό το μοτίβο, τα χαρακτηρίζουν ως ανωμαλίες. Αυτή η τεχνική έχει 2 αδυναμίες.

- Πρώτον, ο αλγόριθμος είναι βελτιστοποιημένος ώστε να βρίσκει κανονικές συμπεριφορές, και όχι ανωμαλίες, οπότε μπορεί να μην είναι τόσο αποτελεσματικός στην ανίχνευση ανωμαλιών.
- Δεύτερον, οι περισσότερες μέθοδοι περιορίζονται σε χαμηλές διαστάσεις δεδομένων, ή και μικρό όγκο δεδομένων, λόγω της υπολογιστικής πολυπλοκότητας που απαιτείται.



Σχήμα 2.7: Pattern-ecg anomaly (αναπαραγωγή από [19])



Σχήμα 2.8: Pattern-shift anomaly (αναπαραγωγή από [19])

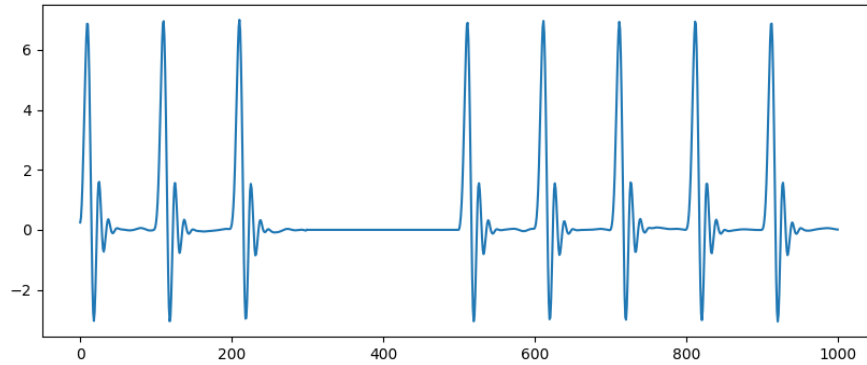
Αντιθέτως, ο *iForest* ακολουθεί μία διαφορετική τακτική, στοχευοποιεί τις ανωμαλίες, αξιοποιώντας 2 βασικές ιδιότητες των ανωμαλιών.

- Πρώτον, οι ανωμαλίες αποτελούν την μειονότητα των δεδομένων.
- Δεύτερον, έχουν χαρακτηριστικά τα οποία διαφέρουν πολύ από τα κανονικά δεδομένα.

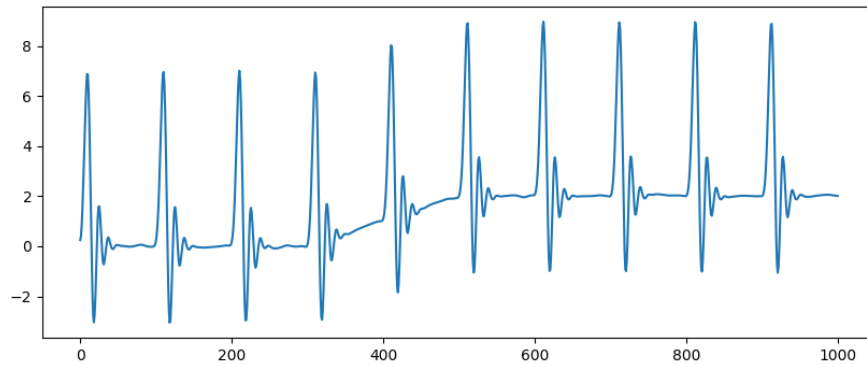
Επειδή οι ανωμαλίες απομονώνονται ευκολότερα από τα κανονικά δεδομένα, βρίσκονται πιο κοντά στην ρίζα του δένδρου. Στο σχήμα 2.14(a), παρατηρούμε πως το σημείο x_i απαιτεί περισσότερες διχοτομήσεις για να απομονωθεί, σε αντίθεση με το σημείο x_0 , όπως φαίνεται στο 2.14(b). Επίσης στο σχήμα 2.14(c), παρατηρούμε πως η μέση απόσταση των σημείων x_0 και x_i από την ρίζα του δένδρου, σταθεροποιείται, με μικρό αριθμό δένδρων ($n \approx 10$), γεγονός που αποδυνκνύει πως ο *iForest* δεν απαιτεί μεγάλο αριθμό δένδρων για να συγκλίνει σε ένα καλό αποτέλεσμα.

Εκτός από τα παραπάνω, ο *iForest* διαφέρει με τους υπόλοιπους αλγόριθμους ανίχνευσης ανωμαλιών και στα παρακάτω:

- Τα χαρακτηριστικά του, του επιτρέπουν να χτίζει μόνο ένα μερικό δένδρο, το οποίο είναι αρκετό για να απομονώσει τις ανωμαλίες. Επίσης δεν χρειάζεται όλα τα δεδομένα, παρά μόνο ένα δείγμα, κάτι το οποίο βοηθάει και στην αντιμετώπιση των φαινομένων του **swamping** και του **masking**.



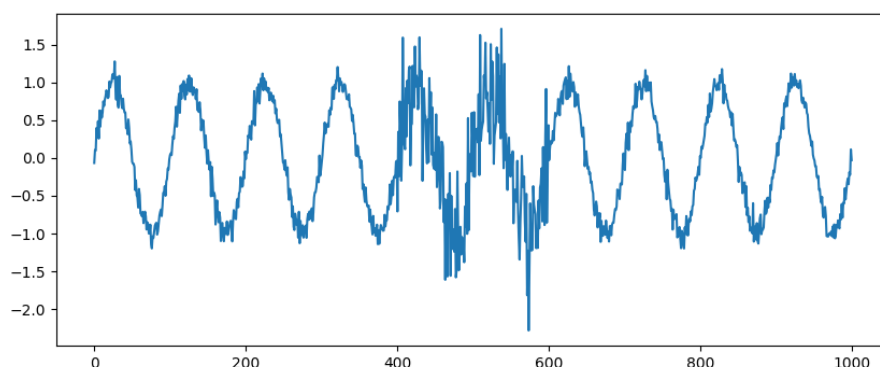
Σχήμα 2.9: Platform anomaly (αναπαραγωγή από [19])



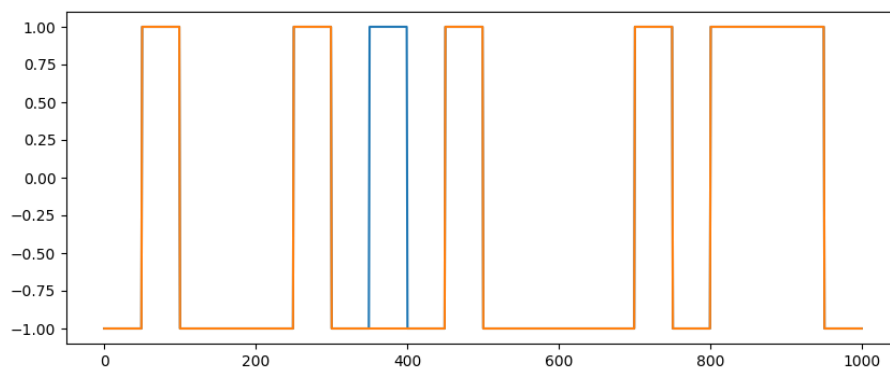
Σχήμα 2.10: Trend anomaly (αναπαραγωγή από [19])

- **Swamping:** Ονομάζεται το φαινόμενο, όπου κανονικά σημεία χαρακτηρίζονται ως ανωμαλίες. Συμβαίνει όταν τα κανονικά σημεία βρίσκονται κοντά σε ανωμαλίες.
- **Masking:** Ονομάζεται το φαινόμενο, όπου πολλά ανώμαλα σημεία δημιουργούν μεταξύ τους ένα cluster, δημιουργώντας την ψευδαίσθηση ενός κανονικού μοτίβου και καθιστώντας δύσκολη την ανίχνευσή τους.
- Δεν χρειάζεται να υπολογίσει αποστάσεις ή πυκνότητες, άρα είναι επεκτάσιμος σε μεγάλους όγκους δεδομένων.
- Έχει γραμμική χρονική πολυπλοκότητα και χαμηλές απαιτήσεις σε μνήμη.
- Τέλος είναι επεκτάσιμος σε μεγάλα σετ δεδομένων με πολλές διαστάσεις.

Οι **υπερπαραμέτροι** του *iForest* είναι ο αριθμός των δένδρων t που θα δημιουργηθούν και ο αριθμός των δειγμάτων δεδομένων ψ που θα χρησιμοποιήσουμε για κάθε δένδρο. Σύμφωνα με τους δημιουργούς του *iForest* [5] μετά από δοκιμές που έγιναν, παρατηρήθηκε ότι για $\psi = 256$ και $t = 100$, δεν υπήρχε μεγάλη βελτίωση του αλγορίθμου στην ανίχνευση ανωμαλιών.



Σχήμα 2.11: Variance anomaly (αναπαράγωγή από [19])

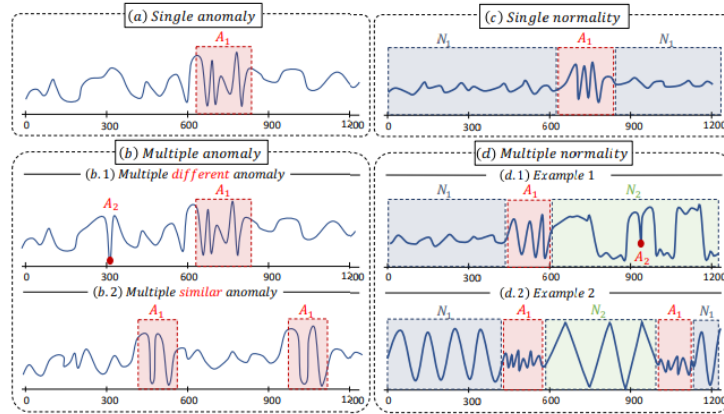


Σχήμα 2.12: Mode correlation anomaly (αναπαράγωγή από [19])

Για την εφαρμογή του *iForest* σε χρονοσειρές, οι δημιουργοί του *TSB-UAD* [3] χρησιμοποίησαν και μία τεχνική συρόμενου παραθύρου, με σκοπό να εξάγουν υπακολουθίες, όπου ο κλασσικός *iForest* χτίζει τα δένδρα του πάνω σε αυτές της υπακολουθίες. Έτσι προκύπτει μία νέα υπερπαράμετρος προς ρύθμιση, το μήκος του παραθύρου L .

2.3.2 NormA

Ο *NormA* [10] είναι ένας αλγόριθμος, ο οποίος χρησιμοποιείται κυρίως όταν θέλουμε να ανιχνεύσουμε ανώμαλες υπακολουθίες μέσα σε μία χρονοσειρά. Οι περισσότεροι αλγόριθμοι οι οποίοι προσπαθούν να εντοπίσουν ανώμαλες υπακολουθίες εντός χρονοσειρών, συνήθως χαρακτηρίζουν ως ανώμαλη την υπακολουθία που έχει την μεγαλύτερη απόσταση από τον πλησιέστερο γείτονά της. Αντιθέτως ο *NormA* λειτουργεί διαφορετικά, πρώτα βρίσκει ένα μοντέλο το οποίο αντικατοπτρίζει την φυσιολογική συμπεριφορά της χρονοσειράς, το οποίο ονομάζει *Normal Model - NN* και έπειτα ορίζει ως ανώμαλες, αυτές που απέχουν περισσότερο από αυτό το μοντέλο.



Σχήμα 2.13: Χρονοσειρές με διαφορετικούς τύπους ανωμαλιών (αναπαράγωγή από [3])

Ένα πρόβλημα το οποίο παρουσιάζεται συχνά στην παραπάνω μέθοδο, είναι όταν μία ανώμαλη υπακολουθία εμφανίζεται μέσα στις χρονοσειρές αρκετές φορές, τόσες ώστε να θεωρείται πλέον ως κανονική. Χαρακτηριστικό παράδειγμα είναι οι αρρυθμίες της καρδιάς ενός ασθενή. Μία τέτοια συμπεριφορά θα μπορούσε να δημιουργήσει cluster από υπακολουθίες αρρυθμιών, και να θεωρούνται πλέον ως κανονικές.

Για να δημιουργήσει το *Normal Model* και να μπορεί να χαρακτηρίζει ως κανονικές ή ανώμαλες τις υπακολουθίες, ο *NormA* δημιουργεί cluster από κανονικές πακολουθίες, δίνοντας στο κάθε cluster ένα βάρος w_i , το οποίο δηλώνει πόσο φυσιολογικό είναι το κάθε cluster. Για τον υπολογισμό του βάρους, ο αλγόριθμος βασίζεται σε 3 χαρακτηριστικά:

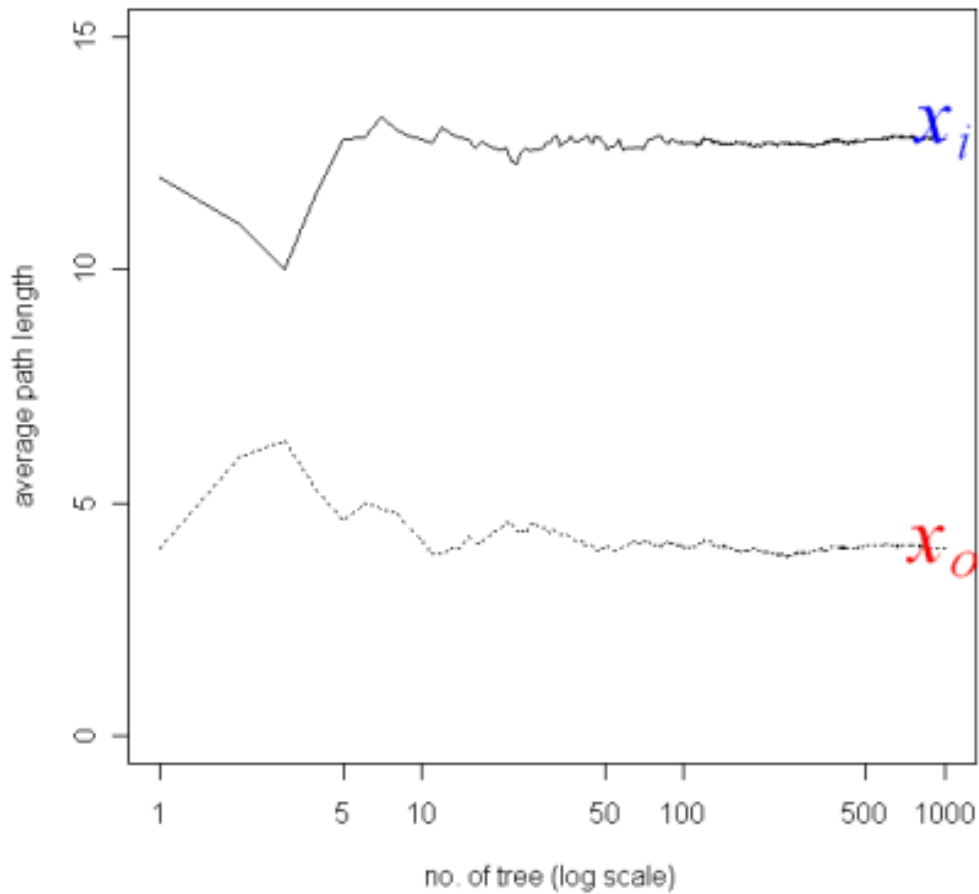
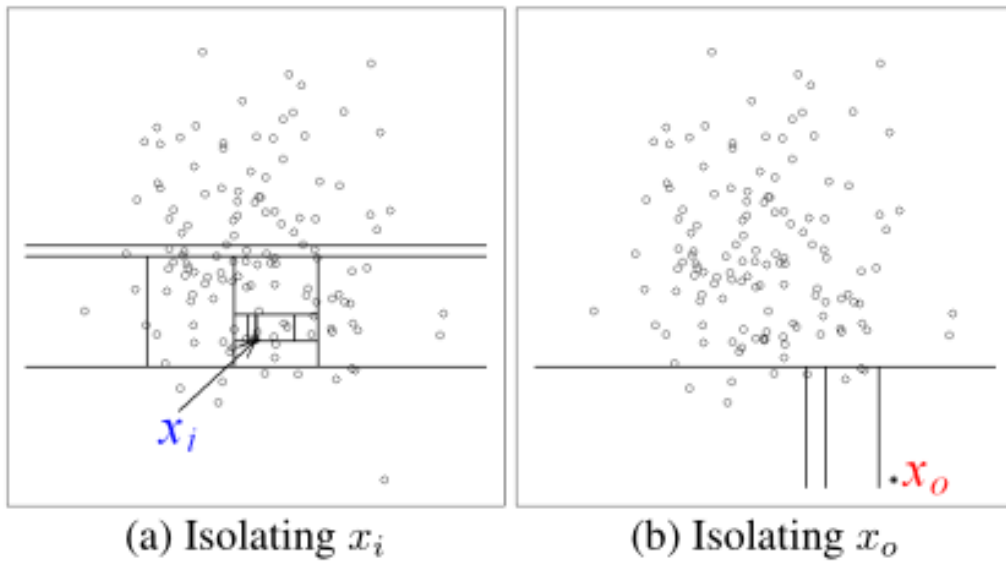
- **Συχνότητα:** Δηλώνει πόσο συχνά εμφανίζεται αυτό το μοτίβο μέσα στην χρονοσειρά.
- **Κάλυψη:** Δηλώνει τι μέρος της συνολικής χρονοσειράς καλύπτει το συγκεκριμένο μοτίβο, δηλαδή σε τι ποσοστό του μήκους της χρονοσειράς εμφανίζεται.
- **Κεντρικότητα:** Δηλώνει πόσο κεντρικό είναι το cluster σε σχέση με τα υπόλοιπα.

Έτσι οι δημιουργοί του *NormA* για να βαθμολογήσουν ένα cluster ακολουθούν τον παρακάτω τύπο:

$$\text{Norm}(c, \mathbb{C}) = \text{Frequency}(c)^2 \times \text{Coverage}(c) \times \text{Centrality}(c, \mathbb{C})$$

Για να επιλεγούν οι υποψήφιες υπακολουθίες για τη δημιουργία του *Normal Model*, υπάρχουν 2 διαφορετικοί τρόποι:

- **Norma-SJ:** Η πρώτη μέθοδος είναι η εξαντλητική, την οποία οι δημιουργοί την ονομάζουν *NormA-SJ*. Εδώ υπολογίζεται η απόσταση της κάθε υπακολουθίας από τον πιο κοντινό γείτονά της, απορρίπτοντας αυτές με την μεγαλύτερη απόσταση, καθώς είναι πιο πιθανό να είναι ανωμαλίες. Το μειονέκτημα αυτής της μεθόδου είναι πως έχει τετραγωνική χρονική πολυπλοκότητα σε σχέση με το μέγεθος της χρονοσειράς.



Σχήμα 2.14: (a) Απομόνωση κανονικού σημείου x_i (b) Απομόνωση ανώμαλου σημείου x_o (c) Μέση απόσταση σημείων x_o και x_i από ρίζα δένδρου, με αυξανόμενο αριθμό δένδρων (αναπαραγωγή από [5])

- **NormA-smpl** Στην δεύτερη μέθοδο, οι ερευνητές επιλέγουν απλά ένα δείγμα από υπακολουθίες οι οποίες είναι μη επικαλυπτόμενες σύμφωνα με ένα ποσοστό r (επισημαίνουν ότι $r = 0.4$ είναι ιδανικό). Το πλεονέκτημα αυτής της μεθόδου είναι πως έχει γραμμική χρονική πολυπλοκότητα σε συνάρτηση με το μέγεθος της χρονοσειράς, έχοντας μικρές διαφορές στην απόδοση του αλγορίθμου.

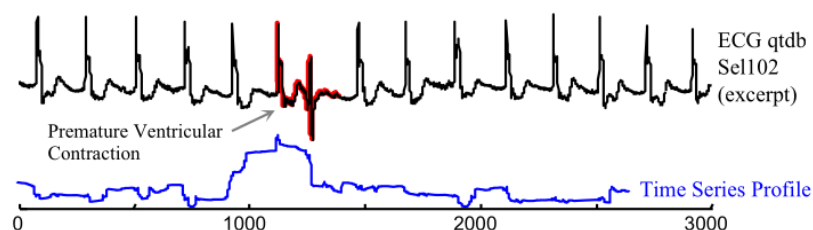
Ένα άλλο φαινόμενο το οποίο μπορεί να επηρεάσει αρκετά τους αλγορίθμους που προσπαθούν να ανιχνεύσουν ανώμαλες υπακολουθίες χρονοσειρών, είναι η *Πολυκανονικότητα*, δηλαδή όταν υπάρχουν 2 ή περισσότερες κανονικές καταστάσεις. Εδώ οι δημιουργείς του NormA χρησιμοποιούν μία επέκτασή του, την οποία ονομάζουν *NormA-mm*, όπου μπορούν να δημιουργηθούν 2 ή παραπάνω μοντέλα ως κανονικά, ώστε να αποφεύγονται τα *False Positives*.

Τέλος, ένα από τα βασικότερα πλεονεκτήματα του NormA είναι πως απαιτεί μόνο μία υπερπαράμετρος αυτορρύθμιση, το αναμενόμενο μήκος της ανωμαλίας l , γεγονός που του δίνει πλεονέκτημα συγκριτικά με άλλους αλγορίθμους.

2.3.3 Matrix Profile - MP

Ο MP [15] δημιουργήθηκε για να λύσει ένα πρόβλημα, γνωστό ως *all-pairs-similarity-search*. Πρακτικά, για κάθε δυνατή υπακολουθία μίας χρονοσειράς, προσπαθεί να εντοπίσει των πιο κοντινό της γείτονα. Ανήκει στην οικογένεια *Time Series subsequences All-Pairs-Similarity-Search* (TSAPSS) αλγορίθμων, οι οποίοι συνήθως έλυναν το παραπάνω πρόβλημα με brute-force μεθόδους, είχαν τετραγωνική πολυπλοκότητα, και δεν μπορούσαν να επεκταθούν σε μεγάλα δεδομένα. Ο MP έχει τα παρακάτω πλεονεκτήματα/χαρακτηριστικά:

- **Είναι ακριβής:** Δεν δίνει ψευδώς αρνητικά αποτελέσματα. Ο ισχυρισμός αυτός αναφέρεται στον υπολογιστικό τομέα της μεθόδου, καθώς ο MP υπολογίζει την απόσταση του κοντινότερου γείτονα με απόλυτη μαθηματική ακρίβεια, αποφεύγοντας προσεγγιστικές τεχνικές, που ενδέχεται να χάσουν δεδομένα. Το παραπάνω, δεν αποτρέπει περιπτώσεις όπου υπάρχουν επαναλαμβανόμενες ανωμαλίες, καθώς ο MP θα τους αποδώσει πολύ μικρό σκορ ανωμαλίας, με συνέπεια να μην μπορεί να τις αναγνωρίσει ως ανωμαλίες.
- **Είναι απλός και δεν απαιτεί ρύθμιση παραμέτρων**
- **Δεν απαιτεί πολύ χώρο αποθήκευσης:** Απαιτεί μόνο $O(n)$ χώρο, σε αντίθεση με άλλους αλγορίθμους που απαιτούν $O(n^2)$.
- **Είναι anytime αλγόριθμος:** Μπορεί να διακοπεί ανά πάσα στιγμή, και να επιστρέψει τα καλύτερα αποτελέσματα μέχρι εκείνη την στιγμή.
- **Είναι αυξηνητικός:** Σε συνεχόμενη ροή δεδομένων, δεν απαιτείται η επανεκτέλεσή του, παρά μόνο η ενημέρωση των αποτελεσμάτων του, με βάση τα καινούργια δεδομένα.
- **Δεν απαιτεί ορισμού κατωφλίου ομοιότητας/απόστασης:** Σε αντίθεση με άλλους αλγορίθμους, δεν απαιτεί να οριστεί κάποιο κατώφλι ομοιότητας/απόστασης, καθώς εκμεταλλεύεται τα *full joins*.



Σχήμα 2.15: Ανίχνευση ανωμαλιών με τον *Matrix Profile* (αναπαραγωγή από [15])

- **Μπορεί να εκμεταλλευτεί το hardware:** Είναι σχεδόν πλήρως παραλληλοποιήσιμος, και μπορεί να εκμεταλλευτεί το hardware, όπως πολλαπλούς πυρήνες, ή ακόμα και GPU, ακόμα και καταναεμημένα συστήματα.
- **Έχει σταθερή χρονική πολυπλοκότητα, η οποία ΔΕΝ είναι ανάλογη του μήκους της υπακολουθίας.**
- **Απαιτεί ντετερμινιστικό χρόνο για να ολοκληρωθεί:** Ένα εξαιρετικά σημαντικό χαρακτηριστικό το οποίο δεν είναι καθόλου συνηθισμένο σε αλγόριθμους αυτού του πεδίου. Δεδομένου μόνο του μήκους της χρονοσειράς, μπορεί να προβλέψει με ακρίβεια τον χρόνο εκτέλεσης που θα χρειαστεί για να ολοκληρωθεί.

Για να κατανοήσει κανείς τον *MP*, πρέπει πρώτα να οριστεί η έννοια του *Distance Profile*. Δεδομένης μίας συγκεκριμένης υπακολουθίας, το *Distance Profile* είναι ένα διάνυσμα στο οποίο αποθηκεύεται η ευκλείδεια απόσταση, εφόσον έχει προηγηθεί Z-κανονικοποίηση, μεταξύ της συγκεκριμένης υπακολουθίας και όλων των άλλων υπακολουθιών. Έτσι το *Matrix Profile*, είναι στην ουσία ένα μονοδιάστατο διάνυσμα, στο οποίο αποθηκεύεται για κάθε υπακολουθία, η ελάχιστη απόστασή της, από το *Distance Profile* της, δηλαδή την απόστασή της από τον πιο κοντινό γείτονά της. Ταυτόχρονα, σε ένα άλλο διάνυσμα, το οποίο ονομάζεται *Matrix Profile Index*, αποθηκεύεται και ο δείκτης της κάθε υπακολουθίας προς τον πιο κοντινό γείτονά της.

Έτσι, μπορούμε να χρησιμοποιήσουμε τον *MP* για δύο αντίθετα προβλήματα:

- **Motif Discovery:** Εντοπισμός των πιο παρόμοιων ζευγών υπακολουθιών. Αυτά τα σημεία, είναι συνεχόμενα και χαμηλά σε ένα διάγραμμα.
- **Discord Discovery:** Εντοπισμός των πιο διαφορετικών ζευγών υπακολουθιών, δηλαδή των ανωμαλιών. Αυτά τα σημεία, σε αντίθεση με πριν, είναι απότομα και δημιουργούν κορυφές σε ένα γράφημα. Όπως φαίνεται και στο σχήμα 2.15, το σημείο που εντοπίστηκε η ανωμαλία στο ηλεκτροκαρδιογράφημα, αντιστοιχεί στο σημείο με την μεγαλύτερη κορυφή στο διάγραμμα του *MP*.

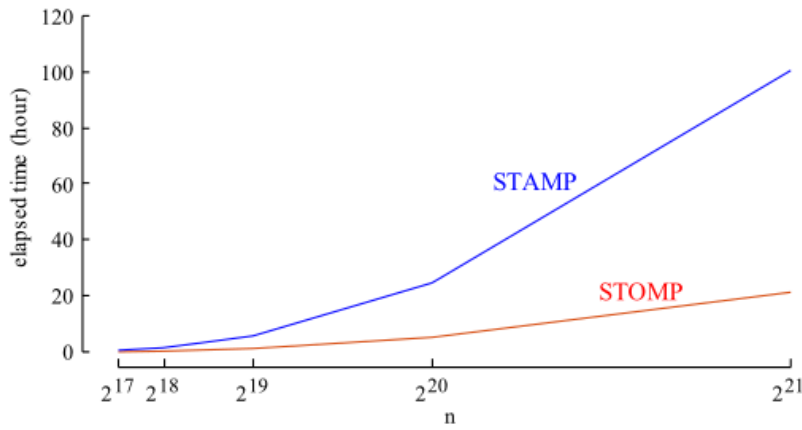
Για τον υπολογισμό του *Distance Profile*, ο *MP* χρησιμοποιεί τον αλγόριθμο *MASS*, ο οποίος βασίζεται στον *Fast Fourier Transform* (FFT). Με αυτόν τον τρόπο, η χρονική πολυπλοκότητα του υπολογισμού του

Distance Profile είναι $O(n \log n)$. Έπειτα, για τον υπολογισμό ολόκληρου του *Matrix Profile*, υπάρχουν 3 διαφορετικές μέθοδοι που ανέπτυξαν οι ερευνητές του *MP*:

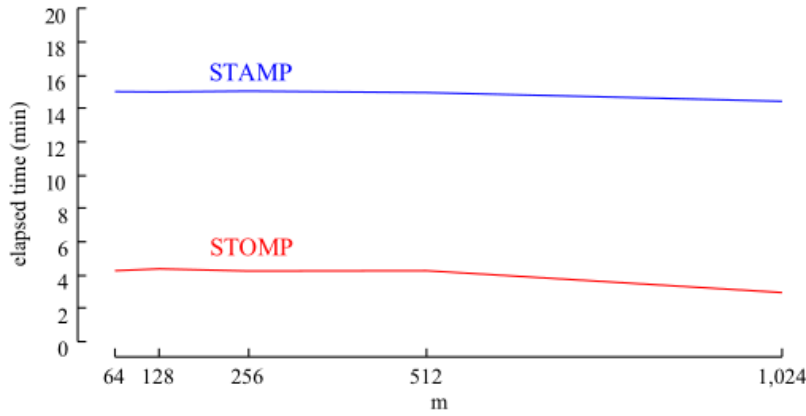
- **STAMP** (*Scalable Time series Anytime Matrix Profile*): Αυτός ο αλγόριθμος, υπολογίζει το *Distance Profile* για κάθε υπακολουθία με τυχαία σειρά. Έτσι, ο χρήστης μπορεί ανά πάσα στιγμή να διακόψει την εκτέλεση του αλγορίθμου, και να πάρει μία προσέγγιση του *Matrix Profile*. Η χρονική πολυπλοκότητα ανέρχεται σε $O(n^2 \log n)$, αλλά με βάση τα πειράματα των δημιουργών, η τελική χρονική πολυπλοκότητα είναι $O(n^2)$. Μία πιθανή εξήγηση για το παραπάνω συμβάν, είναι πως επειδή ο *FFT* είναι πολύ σημαντικός σε πάρα πολλές εφαρμογές, είναι υπερβολικά πολύ βελτιστοποιημένος.
- **STOMP** (*Scalable Time series Ordered-search Matrix Profile*): Είναι μία πιο γρήγορη παραλλαγή του *STAMP* αν ο χρήστης διατίθεται να θυσιάσει το χαρακτηριστικό του *anytime*. Η παραλλαγή αυτή, σαρώνει την χρονοσειρά με αυστηρή σειρά, από την αρχή προς το τέλος. Αυτό είναι σημαντικό, καθώς οι δημιουργοί του *MP* παρατήρησαν πως για να βρεις το εσωτερικό γινόμενο $QT_{j,i}$ όπου Q : η προς εξέταση υπακολουθία, μεταξύ 2 μετατοπισμένων υπακολουθιών κατά μία θέση, αρκεί να υπολογίσεις το προηγούμενο εσωτερικό γινόμενο $QT_{j-1,i-1}$ αφαιρώντας το παλαιότερο σημείο και προσθέτοντας το νεότερο, σε μόλις $O(1)$. Έτσι, η τελική χρονική πολυπλοκότητα του *STOMP* είναι $O(n^2)$.
- **STOMPI** (*Stomp Incremental*): Παραλλαγή του *STOMP*, η οποία δουλεύει για ροές δεδομένων. Δεν επαναυπολογίζει ολόκληρο τον *MP* από την αρχή, παρά μόνο ενημερώνει τα αποτελέσματα με βάση τα καινούργια δεδομένα. Η χρονική πολυπλοκότητα του *STOMPI* είναι $O(n)$, όπου n είναι το μήκος της μέχρι τώρα ακολουθίας. Αυτό συνεπάγεται, πως για κάθε σημείο που προστίθεται, ο αλγόριθμος γίνεται όλο και πιο αργός. Άρα ο καλύτερος τρόπος για να εξετάσουμε την απόδοσή του, είναι να μελετήσουμε τον **Μέγιστο Χρονικό Ορίζοντα (ΜΗΤ)**. Θεωρώντας ένα παράδειγμα, όπου ένα dataset ενημερώνεται κάθε 8 δευτερόλεπτα, ο *STOMPI* θα μπορούσε να τρέξει για 25 χρόνια. Αυτή η εκτίμηση που δίνουν οι ερευνητές είναι, χωρίς να συμπεριλάβουμε τις βελτιστοποιήσεις που συμβαίνουν συνεχώς στο hardware.

Αν και στις περισσότερες περιπτώσεις, οι παραπάνω αλγόριθμοι θα μας κάλυπταν, υπάρχουν και κάποιοι κλάδοι, όπως αυτός των σεσμολόγων, οι οποίοι θα ήθελαν να έχουν απεριόριστο όριο στα δεδομένα τους. Για τέτοιους σκοπούς συγκεκριμένα, οι ερευνητές μετέφεραν τον *STAMP* σε *CUDA C++*, εκμεταλλευόμενοι της τεράστιες δυνατότητες για παράλληλισμό των επεξεργαστών της *NVIDIA*.

Ένα από τα πλεονεκτήματα που αναφέραμε προηγουμένως, είναι πως μπορούμε να γνωρίζουμε εκ των προτέρων, πόσο χρόνο θα χρειαστεί για να εκτελέσουμε τον *MP*. Αυτό το επιτυγχάνουμε κάνοντας αρχικά ένα δοκιμαστικό έλεγχο στο μηχανήμα μας, υπολογίζοντας πόσο χρόνο δ θα κάνει για να υπολογίσει το *MP* σε μία τυχαία χρονοσειρά μεγέθους n . Έπειτα για μία καινούργια χρονοσειρά μεγέθους n_{new} το καινούργιο δ_{new}

Time required for both STAMP and STOMP self-join with $m = 256$, varying n

Σχήμα 2.16: (αναπαραγωγή από [15])

Time required for both STAMP and STOMP self-join with $n = 2^{17}$, varying m

Σχήμα 2.17: (αναπαραγωγή από [15])

υπολογίζεται ως εξής:

$$\delta_{new} = \frac{\delta}{n^2} \times n_{new}^2$$

Σημαντικό είναι να δούμε, τους απαιτούμενους χρόνους για τον *STAMP* και τον *STOMP*, όταν κρατάμε σταθερή την μία από τις 2 παραμέτρους (n, m). Παρατηρούμε στο σχήμα 2.16 πως ο *STOMP* έχει σημαντικά καλύτερους χρόνους εκτέλεσης από τον *STAMP* όσο μεγαλώνει το μέγεθος της χρονοσειράς. Το πιο σημαντικό όμως είναι, όπως φαίνεται στο σχήμα 2.17, πως και οι 2 αλγόριθμοι, είναι σχεδόν σταθεροί ανεξαρτήτως του μεγέθους m της υπακολουθίας. Μάλιστα φαίνεται να υπάρχει μία ελαφρώς φθίνουσα πορεία, όσο μεγαλώνει το m . Αυτό συμβαίνει διότι όσο μεγαλώνει το m , ο αριθμός των συγκρίσεων γίνεται όλο και μικρότερος.

Γενικά ο *MP*, είναι μία από τις ταχύτερες/ακριβέστερες μεθόδους ανίχνευσης μοτίβων/ανωμαλιών, η οποία παρουσιάζει όμως ένα αδύνατο σημείο, το οποίο τονίζουν και οι ερευνητές του. Όταν υπάρχουν επαναλαμβανόμενες ανωμαλίες μέσα στα δεδομένα, μπορεί να θεωρήσει αυτά τα μοτίβα ως φυσιολογικά, διότι βασίζεται αυστηρά στην απόσταση του 1ου

κοντινότερου γείτονα. Για παράδειγμα, αν σε εά ηλεκτροκαρδιογράφημα, ένας ασθενής έχει επαναλαμβανόμενες αρυθμίες, αυτές θα σημανθούν ως κανονικές, κάτι το οποίο δεν είναι επιθυμητό. Το παραπάνω γεγονός επιβεβαιώθηκε και από τους δημιουργούς του *TSB-UAD*.

2.3.4 POLY

Ο αλγόριθμος *POLY* [20] για την εύρεση ανωμαλιών σε χρονοσειρές, σε αντίθεση με άλλους αλγορίθμους οι οποίοι απαιτούν να είναι γνωστό εκ των προτέρων του τύπου του μοντέλου της χρονοσειράς ή είναι αρκετά ευαίσθητοι σε υπερπαράμετρους, αντιμετωπίζει το πρόβλημα με διαφορετική προσέγγιση. Χρησιμοποιώντας την τεχνική της Τοπικής Πολυωνυμικής Προσαρμογής (*Local Polynomial Fitting*), αποφεύγοντας την ανάγκη για παραμέτρους, και με την βοήθεια του *Ασαφή Διαχωρισμού* και της *Θεωρίας Λήψης Αποφάσεων*, μπορεί να εντοπίσει ανωμαλίες σε online χρονοσειρές, αλλά και σημεία αλλαγής (*change points*).

Οι ανωμαλίες και τα σημεία αλλαγής, μπορούν να περιγραφούν μέσω της συνολικής κανονικότητας του συστήματος (*holistic regularity*). Έτσι, οι δημιουργοί του *POLY* ορίζουν σαν ανωμαλία, ένα σημείο το οποίο αποκλίνει σημαντικά από την συνολική κανονικότητα της χρονοσειράς, ενώ εά σημείο αλλαγής, αποτελεί το χρονικό σημείο από το οποίο και μετά, αλλάζει ολόκληρη η συνολική κανονικότητα της χρονοσειράς.

Για να ποσοτικοποιήσουν μαθηματικά αυτήν την συμπεριφορά, εξετάζουν δύο δύο δεσμευμένες πυκνότητες πιθανότητας για κάθε σημείο x_t της χρονοσειράς:

- **Forward Conditional Density:** η οποία υπολογίζεται βάσει ενός παραθύρου δεδομένων του παρελθόντος $p(x_t|x_{t-L}, \dots, x_{t-1})$.
- **Backward Conditional Density:** η οποία υπολογίζεται βάσει ενός παραθύρου δεδομένων του μέλλοντος $p(x_t|x_{t+1}, \dots, x_{t+L})$.

Οι παραδοσιακές μεθόδους, προσπαθούσαν να υπολογίσουν αυτές τις πυκνότητες, προσαρμόζοντας τα δεδομένα τους σε αυστηρά καθολικά παραμετρικά μοντέλα. Κάτι τέτοιο όμως δεν είναι δυνατόν για χρονοσειρές, και έτσι οι δημιουργοί του *POLY* υιοθέτησαν την τεχνική της Τοπικής Πολυωνυμικής Προσαρμογής, όπου για κάθε χρονικό σημείο, απομονώνεται μόνο ένα μικρό τοπικό παράθυρο, όπου ο αλγόριθμος προσπαθεί να ταιριάξει μία απλή τοπική καμπύλη. Βάση αυτής της καμπύλης ο αλγόριθμος υπολογίζει ποιά θα έπρεπε να είναι αναμενόμενη τιμή του σημείου που εξετάζεται.

Έχοντας πλέον την δυνατότητα να προβλέψει την αναμενόμενη τιμή κάθε σημείου μέσω των τοπικών πολυωνύμων, ο αλγόριθμος πρέπει να αποφασίσει αν ένα σημείο είναι ανώμαλο/σημείο αλλαγής ή όχι. Αρχικά, για κάθε σημείο υπολογίζει δύο βαθμολογίες, την *Forward Score* και την *Backward Score*, όπου είναι ίσες με το τετράγωνο της διαφοράς μεταξύ της πραγματικής και της εκτιμώμενης τιμής του σημείου, διαιρούμενη με την διακύμανση των δεδομένων του παραθύρου.

Ωστόσο, η απόφαση για το αν ένα σημείο είναι ανώμαλο/σημείο αλλαγής, δεν γίνεται απλά με βάση κάποιο κατώφλι, αλλά υιοθετείται μία διαφορετική λογική, η οποία πηγάζει από τα *Ασαφή Σύνολα*. Το σύστημα χρησιμοποιώντας τις αρχικές βαθμολογίες, και δύο σταθερές ανοχής,

δεν απορρίπτει ούτε αποδέχεται απόλυτα εά σημείο, αλλά υπολογίζει τον βαθμό συμμετοχής του, σε τέσσερις διαφορετικές καταστάσεις, όπως φαίνονται παρακάτω:

- **FNormalX**: Το σημείο θεωρείται φυσιολογικό σε σχέση με τα δεδομένα του παρελθόντος.
- **BNormalX**: Το σημείο θεωρείται φυσιολογικό σε σχέση με τα δεδομένα του μέλλοντος.
- **NotFNormalX**: Το σημείο δεν θεωρείται κανονικό σε σχέση με τα δεδομένα του παρελθόντος.
- **NotBNormalX**: Το σημείο δεν θεωρείται κανονικό σε σχέση με τα δεδομένα του μέλλοντος.

Έπειτα δημιουργούνται 2 ασαφή σύνολα:

- **Outlier Set**: όπου $Outlier = NotFNormalX \cap NotBNormalX$ και ανήκουν τα ανώμαλα σημεία της χρονοσειράς.
- **Change Point Set**: όπου $Change = NotFNormalX \cap BNormalX$ και ανήκουν τα σημεία αλλαγής της χρονοσειράς.

Οι κύριες ρυθμίσεις που απαιτούνται για τον *POLY* είναι το μέγεθος του παραθύρου h και ο βαθμός του τοπικού πολωνύμου p . Οι δημιουργοί του, προτείνουν ο βαθμός του πολωνύμου να είναι περιττός αριθμός (π.χ. $p = 1, p = 3$), καθώς από πειράματα που έκαναν, παρατήρησαν ότι προσφέρει καλύτερη προσαρμογή στα δεδομένα. Το μέγεθος του παραθύρου h μπορεί να υπολογιστεί δυναμικά, όπου σαν αρχικές τιμές δίνουν $h = [X_{max}^f - X_{min}^f]/(L - 1)$ για την εμπρόσθια εκτίμηση, και $h = [X_{max}^b - X_{min}^b]/(L - 1)$ για την οπίσθια εκτίμηση, αναπροσαρμόζοντάς τας με έναν παράγοντα $C = 1.1$, μέχρι το $MISE(h) \leq \delta$, όπου δ είναι ένα κατώφλι για το $MISE$.

2.3.5 AutoTSAD: Unsupervised Holistic Anomaly Detection for Time Series Data

Εισαγωγή και Πλαισίωση του Προβλήματος

Οι επιστήμονες δεδομένων έχουν αναπτύξει πολλούς διαφορετικούς αλγόριθμους ανίχνευσης ανωμαλιών, με διαφορετικές ικανότητες ο καθένας, όπως την ικανότητα να αναγνωρίζουν επαναλαμβανόμενες ανωμαλίες, ανωμαλίες σε μη περιοδικές χρονοσειρές, ή ανωμαλίες με διαφορετική διάρκεια. Συνήθως για συγκεκριμένο σύνολο δεδομένων και μία συγκεκριμένη εργασία, η εύρεση του κατάλληλου αλγορίθμου, των κατάλληλων παραμετροποιήσεων του και ένα σύνολο από δεδομένα εκπαίδευσης, επιλύουν το πρόβλημα εύρεσης ανωμαλιών αρκετά καλά. Σε περιπτώσεις όμως που δεν γνωρίζουμε την γνώση πεδίου εφαρμογής των δεδομένων, ή δεν υπάρχουν αρκετά δεδομένα εκπαίδευσης, αποτελεί πρόκληση η επιλογή του κατάλληλου αλγορίθμου και των κατάλληλων παραμέτρων για τον κάθε αλγόριθμο ώστε να ανιχνεύσουμε επιτυχώς τις ανωμαλίες.

Λόγω των παραπάνω, προτάθηκε το *AutoTSAD* [1], ένα μη επιβλεπόμενο σύστημα, το οποίο παραμετροποιεί, εκτελεί και συνδυάζει διάφορους εξαιρετικά αποτελεσματικούς αλγόριθμους ανίχνευσης ανωμαλιών. Το σύστημα, παρουσιάζει αυτόματα μία συγκεντρωτική βαθμολογία ανωμαλιών για οποιοδήποτε χρονοσειρά, χωρίς την ανάγκη για δεδομένα εκπαίδευσης ή εξειδίκευση στις παραμέτρους.

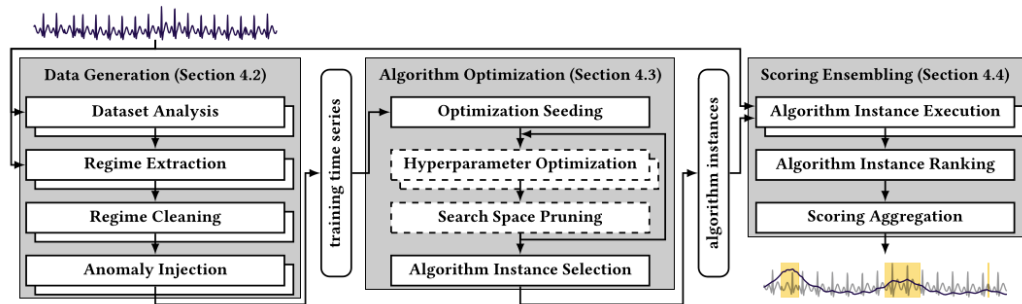
Αρχιτεκτονική Συστήματος

Η αρχιτεκτονική του συστήματος, όπως φαίνεται και στο Σχήμα 2.18, αποτελείται από τρία διαφορετικά δομοστοιχεία, το *data generation*, το *algorithm optimaization* και το *scoring ensembling*. Το κάθε δομοστοιχείο περιλαμβάνει τα παρακάτω βήματα επεξεργασίας:

- Το **Data Generation** δομοστοιχείο, δημιουργεί μία ποικιλία από συνθετικές χρονοσειρές εκπαίδευσης (i) αναλύοντας την χρονοσειρά εισόδου για εντοπισμό κυρίαρχων περιόδων και snippets που σχηματίζουν αντιπροσωπευτικές ακολουθίες, (ii) εξάγοντας διαφορετικών συνεκτικών καθεστώτων (regimes) ως βασικών συμπεριφορών, (iii) καθαρίζοντας τα regimes για να αφαιρέσουν ανώμαλες υπακολουθίες, (iv) και τέλος εισάγοντας διαφορετικών τύπων ακολουθιών για την παραγωγή δεδομένων εκπαίδευσης. Αυτή η διαδικασία οδηγεί στην δημιουργία έως και 120 διαφορετικών χρονοσειρών εκπαίδευσης.
- Το **algorithm optimaization** δομοστοιχείο χρησιμοποιεί τις προεπισημασμένες χρονοσειρές εκπαίδευσης για να δημιουργήσει μία δεξαμενή από βελτιστοποιημένες ποικίλες διαμορφώσεις αλγορίθμων. Αυτό το καταφέρνει μέσω: (i) τροφοδότησης της διαδικασίας βελτιστοποίησης με ελπιδοφόρες ρυθμίσεις υπερπαραμέτρων, (ii) εφαρμογής του Bayesian Optimaization για την αναζήτηση των υπερπαραμέτρων και αλγορίθμων, (iii) αποκοπής μη ελπιδοφόρων ή ήδη πλήρως βελτιστοποιημένων τύπων αλγορίθμων, (iv) αποκοπής πλεονάζουσων χρονοσειρών εκπαίδευσης. Το αποτέλεσμα αυτού του δομοστοιχείου είναι ένα σχετικά μικρό σετ στιγμιότυπων αλγορίθμων, που όλα μαζί επιλύουν επιτυχώς όλες τις χρονοσειρές εκπαίδευσης. Επισημαίνεται ότι η διαδικασία βελτιστοποίησης των υπερπαραμέτρων είναι ένα προαιρετικό βήμα, διότι οι προκαθορισμένες παράμετροι αποδίδουν ήδη καλά στην πράξη.
- Το *scoring ensembling* δομοστοιχείο, συνθέτει την τελική κατάταξη μέσω: (i) της εκτέλεσης των στιγμιότυπων των αλγορίθμων στην χρονοσειρά εισόδου, (ii) της κατάταξης των καλύτερων στιγμιότυπων, (iii) συγκέντρωσης των βαθμολογιών ανωμαλίας με σκοπό την παρουσίασή τους στον χρήστη.

Θεμελιώδης Καινοτομίες και Πλεονεκτήματα του Συστήματος

Αναλύοντας περαιτέρω τα παραπάνω δομοστοιχεία, σημαντικό είναι να σταθούμε σε τρεις καινοτομίες του *AutoTSAD*, οι οποίες αντιμετωπίζουν πρακτικά προβλήματα της ανίχνευσης ανωμαλιών σε χρονοσειρές.



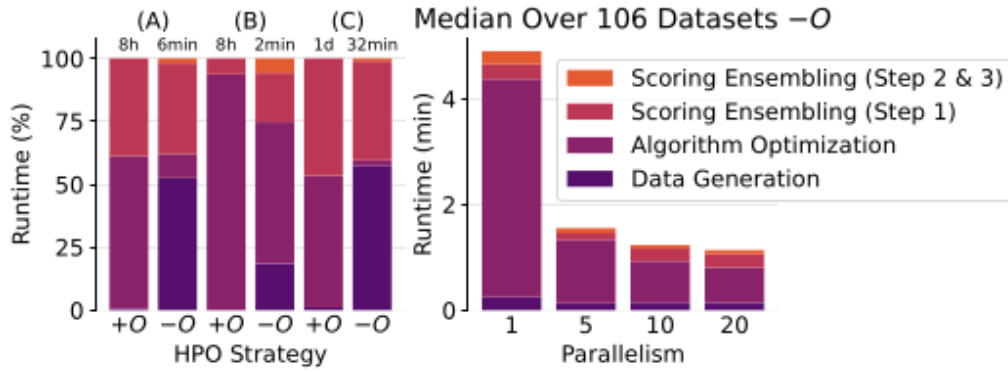
Σχήμα 2.18: Επισκόπηση της αρχιτεκτονικής του AutoTSAD

Έλλειψη Δεδομένων Εκπαίδευσης: Όπως προαναφέρθηκε και στην *Εισαγωγή*, στις περισσότερες μεθόδους Μηχανικής Μάθησης, η απουσία επαρκών επισημειωμένων δεδομένων εκπαίδευσης για την εκπαίδευση των αλγορίθμων, αποτελεί μία από τις σημαντικότερες προκλήσεις. Το *AutoTSAD* μέσω του δομοστοιχείο *Data Generation*, λειτουργεί ως ένα αυτοεπιβλεπόμενο σύστημα. Εντοπίζει τα φυσιολογικά μοτίβα (regimes) της άγνωστης χρονοσειράς και στην συνέχεια εισάγει 9 διαφορετικούς τύπους τεχνητών ανωμαλιών (όπως αυτοί αναφέρθηκαν στο *GutenTAG [19]*). Με αυτόν τον τρόπο δημιουργεί ένα περιβάλλον εκπαίδευσης, προσαρμοσμένο στα χαρακτηριστικά της κάθε χρονοσειράς.

Υπολογιστικό Κόστος και Βελτιστοποίηση Υπερπαραμέτρων: Το *AutoTSAD* είναι ένα ensembling system το οποίο χρησιμοποιεί ένα σετ από οκτώ TSAD Αλγόριθμους οι οποίοι αποδίδουν καλά και έχουν μοναδικά πλεονεκτήματα ο καθένας. Συγκεκριμένα χρησιμοποιεί τους *STOMP*, *k-Means*, *Sub-KNN*, *Sub-LOF*, *Sub-IF*, *GrammarViz*, *Torsk*, *DWT-MLEAD*. Η εύρεση των κατάλληλων υπερπαραμέτρων για αυτούς τους αλγόριθμους γίνεται μέσω τεχνικών Bayesian Optimization με χρήση της βιβλιοθήκης της *Optuna*. Ωστόσο αυτή η διαδικασία είναι εξαιρετικά χρονοβόρα. Η ανάλυση του χρόνου ανά δομοστοιχείο, όπως φαίνεται στο Σχήμα ?? καταδεικνύει τα κάτωθι:

- **Με ενεργοποιημένη την μηχανή της Optuna (O+):** Το βήμα *Algorithm Optimaization* βλέπουμε ότι κυριαρχεί στον συνολικό χρόνο και στα τρία dataset, λόγω των αλληπάλληλων εκτελέσεων που απαιτεί η Bayesian αναζήτησης. Επίσης, η τελική κατάταξη των αλγορίθμων καθυστερεί επίσης αρκετά (όχι όμως τόσο πολύ όσο το βήμα της βελτιστοποίησης), καθώς το σύστημα έχει να συγκρίνει περισσότερα στιγματίσματα αλγορίθμων.
- **Με απενεργοποιημένη την μηχανή της Optuna (O-):** Παραλείποντας το βήμα της βελτιστοποίησης, παρατηρούμε ότι ο συνολικός χρόνος εκτέλεσης μειώνεται δραματικά και στα 3 dataset (έως και 40 φορές), ενώ πλέον το μεγαλύτερο μέρος του χρόνου καταλαμβάνεται για την δημιουργία δεδομένων και την αξιολόγηση των στιγματισμών των αλγορίθμων.

Έτσι, για περιπτώσεις όπου ο χρόνος είναι κρίσιμος, προτείνεται από τους δημιουργούς η απενεργοποίηση αυτού του βήματος, καθώς η χρήση των προκαθορισμένων ρυθμίσεων έχει αποδειχθεί ότι διατηρεί παρόμοια αποτελέσματα στην ακρίβεια της ανίχνευσης.



Σχήμα 2.19: (αριστερά) Κατανομή χρόνου εκτέλεσης του AutoTSAD με βελτιστοποίηση (O+) και χωρίς βελτιστοποίηση (O-) για τα dataset (A) TSB-UAD-artificial-69_2_0.02_15, (B) KDD-TSAD-022, and (C) SAND-SED, (δεξιά) διάμεσοι χρόνοι εκτέλεσης για όλα τα dataset χωρίς βελτιστοποίηση για διαφορετικούς αριθμούς παραλληλισμού

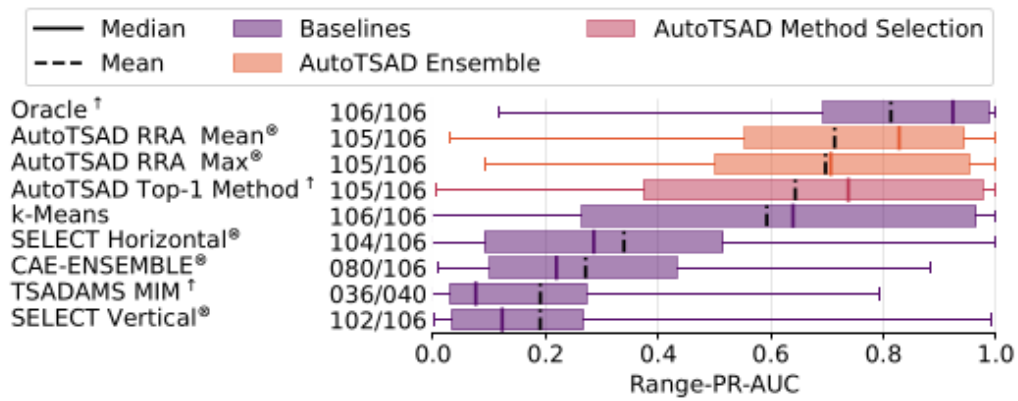
Στρατηγική Ensembling έναντι Μεμονωμένης Επιλογής: Σε αντίθεση με άλλες μεθόδους που προσπαθούν να επιλέξουν έναν μεμονωμένο καλύτερο αλγόριθμο, το *AutoTSAD* βασίζεται στην τεχνική του ensembling. Τα αποτελέσματα των στιγμιότυπων των αλγορίθμων αξιολογούνται με βάση την μετρική *RANGE-PR-AUC*, παρ'όλα αυτά το σύστημα υπολογίζει και τις κλασσικές μετρικές όπως precision, recall κ.ο.κ. Για την τελική συνένωση του score, μπορεί να χρησιμοποιηθεί είτε η *Mean* είτε η *Max Aggregation*. Οι δημιουργοί προτείνουν την χρήση της *Mean Aggregation* καθώς μειώνει την διακύμανση των σφαλμάτων, παράγοντας ένα πιο αξιόπιστο τελικό αποτέλεσμα.

Αξιολόγηση Απόδοσης και Ποιότητα Ανίχνευσης

Για την αξιολόγηση του συστήματος του *AutoTSAD*, οι δημιουργοί του έκαναν πειράματα σε 106 datasets (χωρίς την HPO) για να την συγκρίνουν με τους αλγόριθμους αναφοράς. Η μετρική σύγκρισης ήταν η *RANGE-PR-AUC*. Τα αποτελέσματα των πειραμάτων, όπως φαίνονται στο Σχήμα 2.20, δείχνουν ότι το *AutoTSAD* κατάφερε να ξεπεράσει τόσο τους μεμονωμένους αλγόριθμους, όσο και άλλα υπάρχοντα συστήματα όπως π.χ. το *TSADAMS*. Το σημαντικότερο γεγονός όμως, είναι, πως το *AutoTSAD* κατάφερε να ανταγωνιστεί την ιδεατή περίπτωση του *Oracle*. Τέλος, σημαντικό είναι να αναφερθεί, πως διακρίνεται ο ισχυρισμός που έκαναν οι δημιουργοί, ότι η *Mean Aggregation* επιτυγχάνει καλύτερα αποτελέσματα.

Συμπεράσματα

Το *AutoTSAD* είναι ένα μη-επιβλεπόμενο ensembling σύστημα ανίχνευσης ανωμαλιών, το οποίο δεν απαιτεί κατηγοριοποιημένα δεδομένα εκπαίδευσης. Προσφέρει αρκετές επιλογές για παραμετροποίηση από τον χρήστη. Μπορεί να ανιχνεύσει διαφορετικούς τύπους ανωμαλιών στην



Σχήμα 2.20: Ποιότητα Ανίχνευσης Ανωμαλιών (RANGE-PR-AUC) του *AutoTSAD* συγκριτικά με άλλους baseline αλγόριθμους-συστήματα, σε όλα τα επιτυχώς επεξεργασμένα datasets

ίδια χρονοσειρά. Χρησιμοποιείται σε μονομεταβλητές χρονοσειρές με αρκετή επιτυχία, και οι δημιουργεί του πρόκειται να το επεκτείνουν και για πολυμεταβλητές χρονοσειρές. Πρόκειται για ένα σύστημα, το οποίο βρίσκει ιδιαίτερη εφαρμογή σε περιπτώσεις offline, όπου έχουν ήδη συλλεχθεί τα δεδομένα με σκοπό να αναλυθούν εξονυχιστικά για εύρεση ανωμαλιών (λόγω των μεγάλων χρονικών απαιτήσεων της μηχανής βελτιστοποίησης). Σε περιπτώσεις online, δύναται να εκτελεστεί όμως με κλειστή την μηχανή βελτιστοποίησης.

Κεφάλαιο 3

Πειραματική Διάταξη και Συγκριτική αξιολόγηση

3.1 Γενικά

Σκοπός αυτού του κεφαλαίου είναι να πραγματοποιηθεί μία συστηματική αξιολόγηση των αλγορίθμων που παρουσιάστηκαν στο προηγούμενο κεφάλαιο, σε σχέση με το σύστημα *Auto-TSAD*, κάτω από ένα τυποποιημένο πρότυπο αναφοράς, όπως το *TSB-UAD*.

Τα δεδομένα τα οποία χρησιμοποιήθηκαν για την εκτέλεση των πειραμάτων ήταν από 2 datasets τα οποία είχαν συλλεχτεί από τους δημιουργούς του *TSB-UAD*, το *public_v2* το οποίο υπάρχει διαθέσιμο στο **public_v2 dataset** καθώς και το *Artificial dataset* το οποίο είναι διαθέσιμο στο **artificial dataset**.

Για κάθε dataset έγιναν 2 διαφορετικά πειράματα στο *Auto-TSAD*, ένα με ενεργοποιημένη την μηχανή βελτιστοποίησης *optuna*, η οποία απαιτούσε αρκετό χρόνο για να εκτελέσει ένα πείραμα, και ένα με κλειστή την μηχανή βελτιστοποίησης. Έτσι ο αριθμός των χρονοσειρών που εκτελέστηκαν σε κάθε πείραμα, έχουν όπως στον πίνακα 3.1:

Πίνακας 3.1: Experiments For Auto-TSAD with/out for each dataset

Dataset	Optuna-ON	Optuna-OFF
Public V2	29	3429
Artificial	236	960

3.1.1 Μετρικές Αξιολόγησης

Οι μετρικές που αξιολογήθηκαν ήταν ο χρόνος που απαιτήθηκε, καθώς και το score που πέτυχε η κάθε τεχνική με βάση την μετρική **VUS-ROC**. Ο χρόνος σε συνάρτηση με την *ROC-VUS* είναι σημαντικά, καθώς μας επιτρέπει να αξιολογήσουμε αν η διαφορά στον εντοπισμό των ανωμαλιών είναι αρκετά μεγάλη, για να δικαιολογήσει και το χρονικό κόστος που απαιτεί η μηχανή βελτιστοποίησης *optuna* του *Auto-TSAD*.

3.1.2 Γενικές Παρατηρήσεις

Σαν μία γενική πρώτη παρατήρηση είναι πως το *Auto-TSAD* δεν μπορεί να αποδόσει σωστά χωρίς να έχουμε ενεργοποιήσει την *optuna*. Συγκεκριμένα, το ποσοστό επιτυχίας του, ώστε να μπορέσει να βρει έστω και μία ανωμαλία, ήταν πολύ χαμηλό όταν δεν ήταν ενεργοποιημένη η *optuna*. Τα αποτελέσματα φαίνονται στο Σχήμα 3.1. Παρατηρούμε τόσο στο *Public* όσο και στο *Artificial* dataset, το ποσοστό επιτυχίας ανίχνευσης έστω και μίας ανωμαλίας αγγίζει το 100% όταν η *optuna* είναι ενεργοποιημένη, ενώ αντιθέτως τα ποσοστά επιτυχίας είναι πολύ πιο χαμηλά με τιμές 31,2% για το *Public* και 9,2% για το *Artificial* dataset. Συγκεκριμένα, χωρίς την *optuna* ενεργοποιημένη, το σύστημα *Auto-TSAD* δεν μπόρεσε να πάρει κανένα αποτέλεσμα, για καμία μετρική. Το σύστημα *Auto-TSAD* υπολογίζει την απόδοση στις παρακάτω μετρικές:

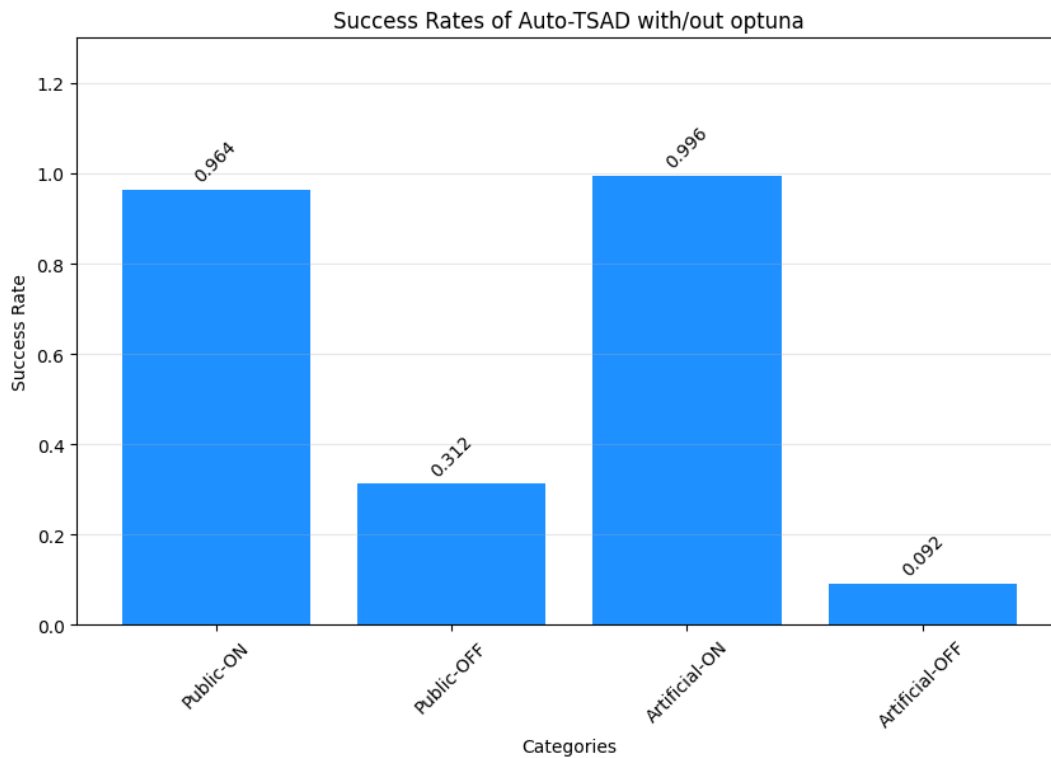
- PrAUC
- RocAUC
- RangePrAUC
- RangeRocAUC
- RangePrVUS
- RangeRocVUS
- RangePrecision
- RangeRecall
- RangeFScore
- Precision
- Recall
- FScore
- PrecisionAtK

3.1.3 TODO

TODO ΓΙΑΤΙ ΔΕΝ ΕΠΙΣΤΡΕΦΕΙ ΟΜΩΣ ΑΠΟΤΕΛΕΣΜΑΤΑ? ΔΕΝ ΕΙΝΑΙ ΠΕΡΙΟΔΙΚΑ??

3.2 Public Dataset

Σε αυτό το κεφάλαιο θα συγκρίνουμε τα αποτελέσματα του *Auto-TSAD* με τις άλλες μετρικές *IForesit - POLY - MP*. Για το *Auto-TSAD*, με ανοιχτή την μηχανή βελτιστοποίησης *optuna*, επιλέχτηκε τυχαία μία χρονοσειρά από κάθε dataset, ενώ με κλειστή την μηχανή *optuna*, ο *Auto-TSAD* έτρεξε στο σύνολο των χρονοσειρών. Οι μετρικές *IForest - POLY - MP* εκτελέστηκαν μόνο στις ίδιες επιλεγμένες χρονοσειρές που έτρεξε και ο *Auto-TSAD*.



Σχήμα 3.1: Ποσοστά επιτυχίας ανίχνευσης τουλάχιστον μίας ανωμαλίας του Auto-TSAD

3.2.1 Τα dataset

Τα dataset που επιλέχτηκαν από τους δημιουργούς του *Auto-TSAD*, καλύπτουν μία ευρεία γκάμα από διαφορετικούς τομείς, με διαφορετικούς τύπους ανωμαλιών. Πιο συγκεκριμένα:

- *Genesis*: Προέρχεται από έναν φορητό επιδείκτη τοποθέτησης και μεταφοράς, ο οποίος χρησιμοποιεί μία δεξανμενή αέρα για να τροφοδοτήσει ρομποτικές μονάδες λαβής και αποθήκευσης. Περιέχει ακολουθίες ανωμαλιών.
- *WSD*: Αφορά χρονοσειρές που συλλέγουν δεδομένα από μεγάλες εταιρίες του Διαδικτύου, και περιέχουν Βασικούς Δείκτες Απόδοσης. Περιέχει ακολουθίες ανωμαλιών.
- *MSL*: Συλλογή δεδομένων της NASA που προέρχονται από τους αισθητήρες του Rover Curiosity από τον Άρη. Περιέχει ακολουθίες ανωμαλιών.
- *Stock*: Αφορά δεδομένα από αγορές μετοχών. Περιέχει ανωμαλίες σημείων.
- *Daphnet*: Ιατρικά δεδομένα που σχετίζονται με την νόσο του Parkinson. Καταγράφει ενδείξεις από 3 αισθητήρες επιτάχυνσης οι οποίοι είναι τοποθετημένοι στο ισχύο και στο πόδι ασθενών, και εντοπίζουν το φαινόμενο freezing of gate (FoG) κατά τη διάρκεια εργασιών περπατήματος. Περιέχει ακολουθίες ανωμαλιών.

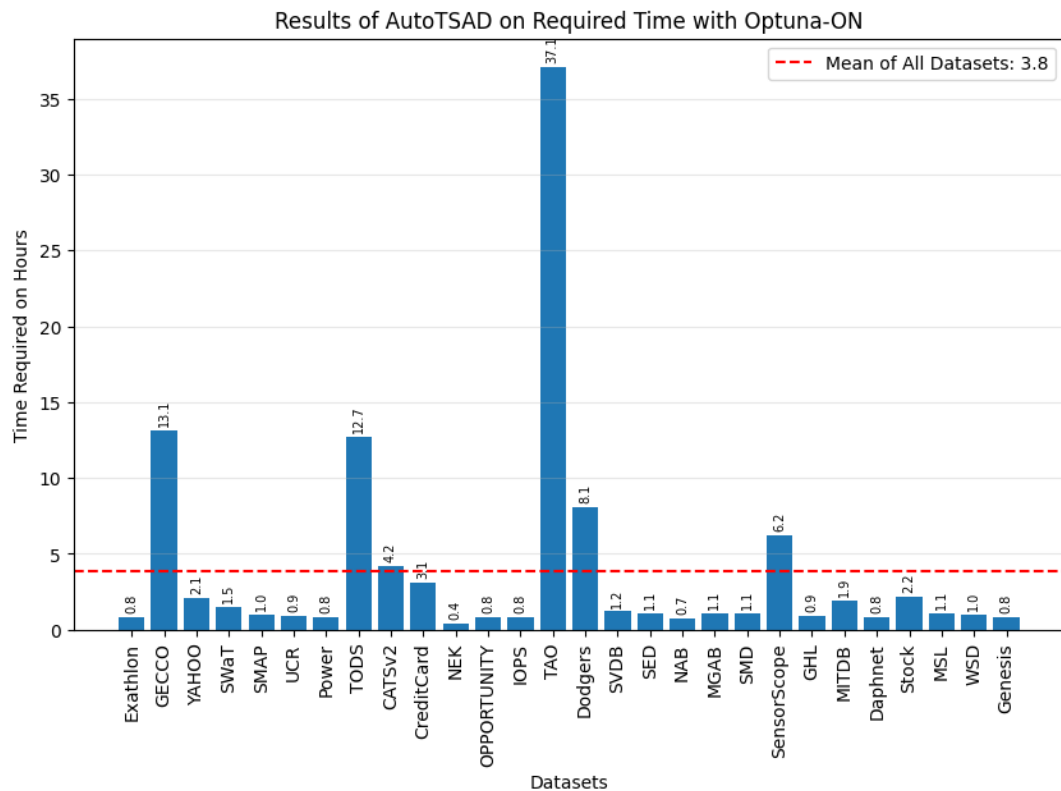
- *MITDB*: Αφορά δεδομένα που συλλέχτηκαν από 47 ασθενής, και περιέχει 48 αποσπάσματα μισής ώρας ηλεκτροκαρδιογραφημάτων διπλού καναλιού, και προσπαθούν να εντοπίσουν καρδιακές αρυθμίες. Περιέχει ακολουθίες ανωμαλιών.
- *GHL*: Αφορά δεδομένα τα οποία συλλέγονται από 3 δεξαμενές σε έναν βρόγχο θέρμανσης πετρελαίου. Καταγράφει την θερμοκρασία και την στάθμη τους. Οι ανωμαλίες αντιστοιχούν σε αλλαγές στην μέγιστη θερμοκρασία ή την συχνότητα της αντλίας. Πρόκειται για ακολουθίες ανωμαλιών.
- *Sensorscope*: Μία συλλογή από περιβαλλοντικά δεδομένα που αφορούν θερμοκρασία, υγρασία, ηλιακή ακτινοβολία κ.ο.κ. Πρόκειται για ακολουθίες ανωμαλιών.
- *SMD*: Δεδομένα καταγραφής λειτουργίας 28 διαφορετικών server, 3 διαφορετικών ομάδων, διάρκειας 5 εβδομάδων από μεγάλη εταιρία Διαδικτύου. Περιέχει ακολουθίες ανωμαλιών.
- *MGAB*: Αποτελείται από χρονοσειρές Mackey-Grass, η οποίες παρουσιάζουν χαοτική συμπεριφορά, κάνοντας πολύ δύσκολο το έργο ανίχνευσης ανωμαλιών με γυμνό μάτι. Περιέχει ακολουθίες ανωμαλιών.
- *NAB*: Σύνθετα δεδομένα από πραγματικές και τενητές χρονοσειρές, οι οποίες προέρχονται από διάφορους server, όπως AWS, αναφορές στο *X* σε μεγάλες εισηγμένες εταιρίες κ.ο.κ. Περιλαμβάνει ακολουθίες ανωμαλιών.
- *SED*: Αφορά δεδομένα από δίσκους κινητήρων της NASA σε διάφορες φάσης λειτουργίας, σε σταθερή ταχύτητα 3000rpm. Αφορά ακολουθίες ανωμαλιών.
- *SVDBM*: Περιλαμβάνει 78 ιατρικά καρδιογραφήματα, ημίωρης διάρκειας, επιλεγμένες ειδικά για να συμπληρώσουν το *MITB*. Περιέχει ακολουθίες ανωμαλιών.
- *Dodgers*: Δεδομένα καταγραφής κίνησης στον αυτοκινητόδρομο 101 του LA, μετά το τέλος αγώνα της ομάδας μπέιζμπολ Dodgers. Περιέχει ακολουθίες ανωμαλιών.
- *TAO*: Δεδομένα που αφορούν μετεωρολογικές και ωκεανογραφικές μετρήσεις. Περιέχει και μεμονωμένες ανωμαλίες αλλά και ακολουθίες ανωμαλιών.
- *IOPS*: Δεδομένα που αφορούν την ποιότητα των διαδικτυακών υπηρεσιών και την υγεία ενός μηχανήματος. Περιέχει ακολουθίες ανωμαλιών.
- *OPPORTUNITY*: Αφορά αισθητήρες οι οποίοι συλλέγουν δεδομένα κατά την ανθρώπινη δραστηριότητα. Περιέχει ακολουθίες ανωμαλιών.
- *NEK*: Περιέχει δεδομένα από διαδικτυακές συσκευές παραγωγής. Περιέχει τόσο σημειακές ανωμαλίες όσο και ακολουθίες ανωμαλιών.

- *CreditCard*: Αφορά δεδομένα συναλλαγών μέσω πιστωτικών καρτών. Προσπαθεί να εντοπίσει απάτες. Περιέχει μεμονωμένες ανωμαλίες καθώς και ακολουθίες ανωμαλιών.
- *CATSV2*: Περιλαμβάνει δεδομένα από εντολές συστήματος, εξωτερικά ερεθίσματα και μετρήσεις τηλεμετρίας, στις οποίες έχουν εισαχθεί 200 ανωμαλίες. Περιέχει ακολουθίες ανωμαλιών.
- *TODS*: Συνθετικό dataset το οποίο περιέχει διαφορετικού τύπου δεδομένων για την αξιολόγηση αλγορίθμων εύρεσης ανωμαλιών. Περιέχει τόσο μεμονωμένες ανωμαλίες καθώς και ακολουθίες ανωμαλιών.
- *Power*: Καταγράφει δεδομένα ζήτησης/κατανάλωσης ενέργειας. Περιέχει ακολουθίες ανωμαλιών.
- *UCR*: Συλλογή από διάφορες μονοσδιάστατες χρονοσειρές, από διάφορους τομείς, στις οποίες οι περισσότερες ανωμαλίες έχουν εισηχθεί τεχνητά. Περιέχει και μεμονωμένες ανωμαλίες αλλά και ακολουθίες ανωμαλιών.
- *SMAP*: Πραγματικά δεδομένα από καταγραφές και μετρήσεις αεροδιαστημικών συστημάτων και δορυφόρων. Περιέχει ακολουθίες ανωμαλιών.
- *SWaT*: Περιλαμβάνει δεδομένα τα οποία προέρχονται από 51 αισθητήρες και ενεργοποιητές, όπου αναλύουν το νερό. Οι ανωμαλίες αντιπροσωπεύουν λάθος ενδείξεις στις μετρήσεις των αισθητήρων λόγω κυβερνοεπιθέσεων. Περιέχει ακολουθίες ανωμαλιών.
- *YAHOO*: Περιλαμβάνει δεδομένα τα οποία προέρχονται από τα εργαστήρια της YAHOO και αφορά πραγματικές και συνθετικές χρονοσειρές, βασισμένες στην πραγματική κίνηση παραγωγής διάφορων συστημάτων της εταιρίας. Περιέχει τόσο μεμονωμένες ανωμαλίες, αλλά και ακολουθίες ανωμαλιών.
- *GECCO*: Dataset το οποίο αφορά στην ποιότητα του νερού, για online ανίχνευση ανωμαλιών. Περιέχει ακολουθίες ανωμαλιών.
- *EXATHLON*: Δεδομένα από πραγματικά ίχνη δεδομένων, τα οποία συλλέχτηκαν επί 2,5 μήνες από ένα Spark cluster. Για κάθε ανωμαλία παρέχονται ακριβές ετικέτες τόσο για το χρονικό διάστημα της βασικής αιτίας, αλλά και για το χρονικό διάστημα των επιπτώσεων. Περιέχει ακολουθίες ανωμαλιών.

3.2.2 Χρόνοι εκτέλεσης

Τα πειράματα εκτελέστηκαν σε τρία μηχανήματα με λειτουργικό σύστημα Fedora Linux 43. Το πρώτο μηχάνημα διέθετε επεξεργαστή Intel Core i5-10400 (6 πυρήνες, 4GHz), κάρτα γραφικών AMD Radeon RX 5700 XT και 16GB μνήμης RAM DDR4. Τα υπόλοιπα δύο μηχανήματα ήταν ίδια μεταξύ τους, με επεξεργαστή Intel Core i5-9500T (6 πυρήνες, 2.2GHz), ενσωματωμένη κάρτα γραφικών Intel UHD Graphics 630 και 16GB μνήμης RAM DDR4.

Όπως αναφέρθηκε και προηγουμένως, σημαντικό είναι να εξετάσουμε τους χρόνους εκτέλεσης. Η μηχανή βελτιστοποίησης της *optuna*, που χρησιμοποιεί το *Auto-TSAD* απαιτεί αρκετό χρόνο για να υπολογίσει και να βρει τις καλύτερες υπερπαραμέτρους για τον κάθε αλγόριθμο, ώστε να καταλήξει στην χρήση ενός αλγόριθμου με τις καλύτερες δυνατές υπερπαραμέτρους. Τα αποτελέσματα φαίνονται συγκεντρωτικά στο Σχήμα 3.2. Ο μέγιστος χρόνος εκτέλεσης που παρατηρήθηκε, ήταν 37,1 ώρες (37 ώρες και 6 λεπτά) για το dataset *TAO*, ενώ ο ελάχιστος χρόνος ήταν 0,4 ώρες (24 λεπτά). Επίσης η μέση τιμή είναι 3,84 ώρες (3 ώρες και 50 λεπτά) και η τυπική απόκλιση είναι 7,35 ώρες (7 ώρες και 21 λεπτά). Συγκεντρωτικά όλα τα στατιστικά στοιχεία φαίνονται στον πίνακα.



Σχήμα 3.2: Χρόνοι εκτέλεσης για κάθε dataset με ανοιχτή την μηχανή βελτιστοποίησης *optuna*, σε ώρες.

Πίνακας 3.2: Στατιστικά χρόνου εκτέλεσης με την *optuna* στο *publicv2* dataset

Measure	Value
Mean	3.84
std	7.35
min	0.40
25%	0.80
50%	1.10
75%	2.43
max	37.10

Ενώ το 75% των dataset εκτελούντε μέχρι 2 ώρες και 26 λεπτά, υπάρχουν και κάποια dataset όπως το *GECCO*, *TODS*, *TAO*, *DODGERS* και *SensorScope* για τα οποία απαιτήθηκε πολύ παραπάνω χρόνος εκτέλεσης. Αυτά τα δεδομένα, δεν παρουσιάζουν όλα μαζί κάποιο ενιαίο χαρακτηριστικό, το οποίο να δικαιολογεί συνολικά τον τόσο αυξημένο χρόνο εκτέλεσής του. Παρ' όλα αυτά, μία πιθανή εξήγηση είναι η εξής:

- Το *GECCO* και το *DODGERS* έχουν σαν μέσο μήκος χρονοσειράς, 138.000 και 50.400 σημεία αντίστοιχα. Το *Auto-TSAD* επειδή μπορεί να εκτελέσει τον κάθε αλγόριθμο μέχρι και 800 φορές για την εύρεση των καλύτερων υπερπαραμέτρων, σε συνδιασμό με τον μεγάλο όγκο των δεδομένων, μπορεί να εκτοξεύσει τους χρόνους εκτέλεσης.
- Το *TAO* και το *TODS*, ενώ δεν έχουν τόσο μεγάλο μήκος χρονοσειρών, φαίνεται ότι έχουν σχετικά υψηλό ποσοστό ανωμαλιών, με 9,4% και 6,3% αντιστοίχως, καθώς επίσης οι τύποι των ανωμαλιών που περιέχουν είναι και μεμονωμένες ανωμαλίες αλλά και ακολουθίες ανωμαλιών.
- Τέλος το *SensorScope* αλλά και το *DODGERS* παρουσιάζουν πολύ υψηλό ποσοστό ανωμαλιών με 22,5% και 11,14% αντίστοιχα.

Γενικά, φαίνεται πως αλγόριθμοι οι οποίοι δεν μπορούν να κλιμακώσουν σωστά σε μεγάλο σετ δεδομένων, ή με πολλές ανωμαλίες, δυσκολεύουν πάρα πολύ την μηχανή της Optuna και πολλές φορές αναγκάζεται να τρέξει σχεδόν όλες τις επαναλήψεις για όλους τους αλγορίθμους που χρησιμοποιποιεί, καθώς δεν μπορεί να βρει 10 σετ υπερπαραμέτρων με απόδοση μεγαλύτερη του 95% για να σταματήσει την λειτουργία της.

Οι χρόνοι εκτέλεσης χωρίς ανοιχτή την μηχανή *Optuna* είναι πολύ μικρότεροι, με μέσο χρόνο εκτέλεσης τα 3 λεπτά και τυπική απόκλιση 12 λεπτών. Ενδιαφέρον εύρημα αποτελεί ο μέγιστος χρόνος εκτέλεσης, ο οποίος ανέρχεται σε 6 ώρες και 41 λεπτά, ο οποίος ανήκει στην χρονοσειρά με όνομα *LTDB\14157_col_0* η οποία βρέθηκε να έχει 9.5×10^6 σημεία. Τα αποτελέσματα βρίσκονται στον πίνακα 3.3. Αν επαναυπολογίσουμε τα καινούργια στατιστικά, χωρίς αυτήν την ακραία τιμή, τότε η καινούργια μέση τιμή είναι ίση με 167 sec, ενώ η καινούργια τυπική απόκλιση είναι 74 sec, όπως φαίνεται και στον πίνακα 3.4.

Πίνακας 3.3: Χρόνοι εκτέλεσης σε sec στο dataset public v2, χωρίς ενεργή την μηχανή βελτιστοποίησης

Statistics	Runtimes in seconds
mean	190
std	736
median	54
min	3
max	24109

Όσον αφορά το *TSB-UAD*, εξετάστηκαν οι χρόνοι εκτέλεσης των αλγορίθμων *IForest* - *POLY* και *MP*, καθώς δεν υπήρχε πρόσβαση στον αλγόριθμο *NORMA* από τους δημιουργούς του *TSB-UAD*. Η εφαρμογή τους

Πίνακας 3.4: Χρόνοι εκτέλεσης σε sec στο dataset public v2, χωρίς ενεργή την μηχανή βελτιστοποίησης και χωρίς ακραίες τιμές

Statistics	Runtimes in seconds
mean	167
std	74

έγινε πάνω στις ίδιες χρονοσειρές οι οποίες επιλέχτηκαν από το *Auto-TSAD* με ενεργοποιημένη την μηχανή *Optuna* για να βγουν όσο το δυνατό πιο αντικειμενικά αποτελέσματα. Ο υπολογισμός του χρόνου για κάθε αλγόριθμο, αφορά από την εκπαίδευση των μοντέλων μέχρι και την εξαγωγή των score, ενώ δεν συμπεριλήφθηκε ο υπολογισμός των μετρικών. Για την ανάπτυξη του κώδικα, χρησιμοποιήθηκε το notebook το οποίο υπάρχει στο github repository των δημιουργών του *TSB-UAD*, το οποίο βρίσκεται στον σύνδεσμο [Anomaly detection for time series from TSB-UAD authors](#). Ο κώδικας για την εκτέλεση των τριών αλγορίθμων έχει όπως στα Σχήματα 3.3 έως 3.5. **TODO ΜΑΛΛΟΝ ΠΡΕΠΕΙ ΝΑ ΓΡΑΨΩ ΚΑΙ ΤΙ ΚΑΝΕΙ Ο ΚΩΔΙΚΑΣ ΕΔΩ !!!!!!!!!!!!!!!!!!!!!!!**

```
def runIForest(data, label, slidingWindow, X_data):
    start_time = time.time()
    clf = IForest(n_jobs=1)
    clf.fit(X_data)
    score = clf.decision_scores_
    end_time = time.time()
    score = MinMaxScaler(feature_range=(0,1)).fit_transform(score.reshape(-1,1)).ravel()
    score = np.array([score[0]]*math.ceil((slidingWindow-1)/2) + list(score) + [score[-1]]*((slidingWindow-1)//2))
    results = get_metrics(score, label, metric='all', slidingWindow=slidingWindow)
    runtime = end_time - start_time
    return results, runtime
```

Σχήμα 3.3: Κώδικας για την εκτέλεση του IForest μέσω του TSB-UAD

```
def runPOLY(data, label, slidingWindow):
    start_time = time.time()
    clf = POLY(power=3, window=slidingWindow)
    clf.fit(data)
    measure = Fourier()
    measure.detector = clf
    measure.set_param()
    clf.decision_function(measure=measure)
    score = clf.decision_scores_
    end_time = time.time()
    score = MinMaxScaler(feature_range=(0,1)).fit_transform(score.reshape(-1,1)).ravel()
    runtime = end_time - start_time
    metrics = get_metrics(score, label, metric='all', slidingWindow=slidingWindow)
    return metrics, runtime
```

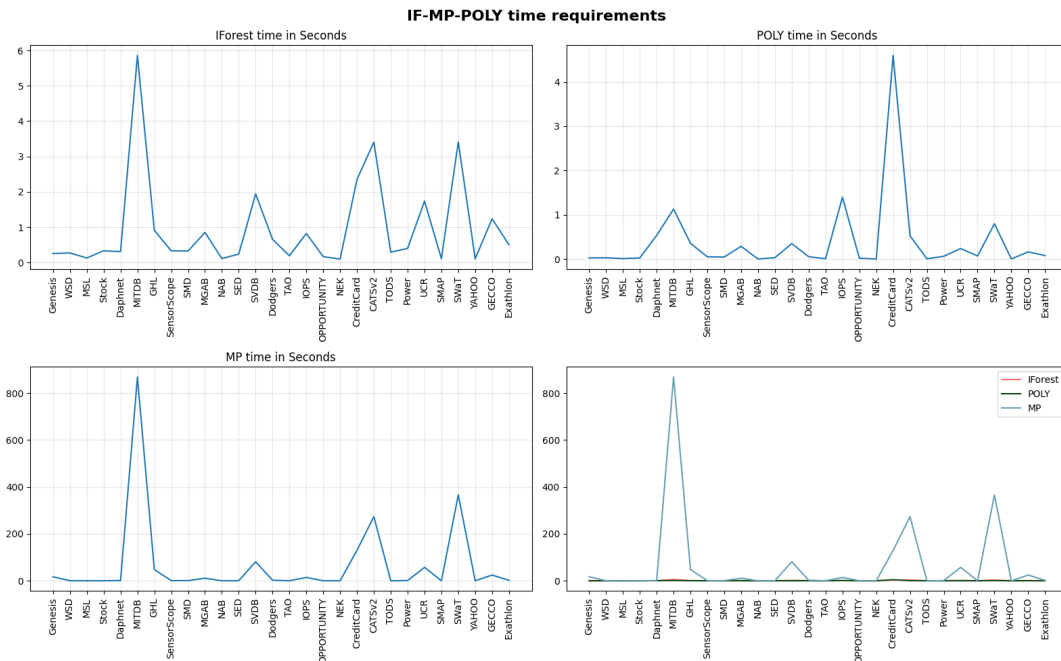
Σχήμα 3.4: Κώδικας για την εκτέλεση του POLY μέσω του TSB-UAD

Οι απαιτήσεις εκτέλεσης αυτών των αλγορίθμων ήταν πολύ μικρές, της τάξης των 5-6 δευτερολέπτων μέγιστο για τους *IForest* και *POLY*, ενώ για τον *MP* κάποιες φορές απαιτήθηκε παραπάνω χρόνος, με μέγιστες τιμές μέχρι και 800 δευτερόλεπτα. Σαν γενική εντύπωση είναι πως το *TSB-UAD* χρειάστηκε αρκετά μικρότερο χρόνο (της τάξης των δευτερολέπτων μέχρι και κάποιων λεπτών) σε αντίθεση με το *Auto-TSAD* το

```
def runMP(data, label, slidingWindow):
    slidingWindow = min(slidingWindow, len(data) // 2)
    start_time = time.time()
    clf = MatrixProfile(window=slidingWindow)
    clf.fit(data)
    score = clf.decision_scores_
    end_time = time.time()
    score = MinMaxScaler(feature_range=(0,1)).fit_transform(score.reshape(-1,1)).ravel()
    score = np.array([score[0]]*math.ceil((slidingWindow-1)/2) + list(score) + [score[-1]]*((slidingWindow-1)/2))
    min_len = min(len(score), len(label))
    score = score[:min_len]
    label = label[:min_len]
    metrics = get_metrics(score, label, metric='all', slidingWindow=slidingWindow)
    runtime = end_time - start_time
    return metrics, runtime
```

Σχήμα 3.5: Κώδικας για την εκτέλεση του MP μέσω του TSB-UAD

οποίο σε εξαιρετικές περιπτώσεις απαιτούσε ακόμα και μέρες για να παράξει αποτελέσματα. Στο σχήμα 3.6 παρουσιάζονται οι χρονικές απαιτήσεις του κάθε αλγορίθμου που εκτελέστηκαν μέσω του *TSB-UAD*. Παρότι οι αλγόριθμοι που έτρεχαν στο *TSB-UAD* απαιτούσαν σχεδόν μηδενικό χρόνο σε σύγκριση με *Auto-TSAD*, είναι σημαντικό πως, όπως θα δούμε παρακάτω, όλοι κατάφεραν να παράξουν αποτελέσματα, δηλαδή να ανιχνεύσουν ανωμαλίες (το ποσοστό αποτυχίας ανίχνευσης ανωμαλιών του *Auto-TSAD* με κλειστή την μηχανή της Optuna ήταν $\approx 69\%$).



Σχήμα 3.6: Κώδικας για την εκτέλεση του MP μέσω του TSB-UAD. Πάνω αριστερά διακρίνουμε τις χρονικές απαιτήσεις του IForest. Πάνω δεξιά της χρονικές απαιτήσεις του POLY. Κάτω αριστερά διακρίνουμε τις χρονικές απαιτήσεις του MP. Κάτω δεξιά διακρίνουμε συγκριτικά τις χρονικές απαιτήσεις όλων των αλγορίθμων.

3.2.3 Αποτελέσματα ROC-VUS

Θα εξετάσουμε τα αποτελέσματα της μετρικής *ROC-VUS*:

1. Με ανοιχτή την Optuna και μεταξύ των αλγορίθμων που τρέχουμε στο *TSB-UAD*
2. Με κλειστή την Optuna και μεταξύ των αλγορίθμων που τρέχουμε στο *TSB-UAD*
3. Μεταξύ όλων των αποτελεσμάτων

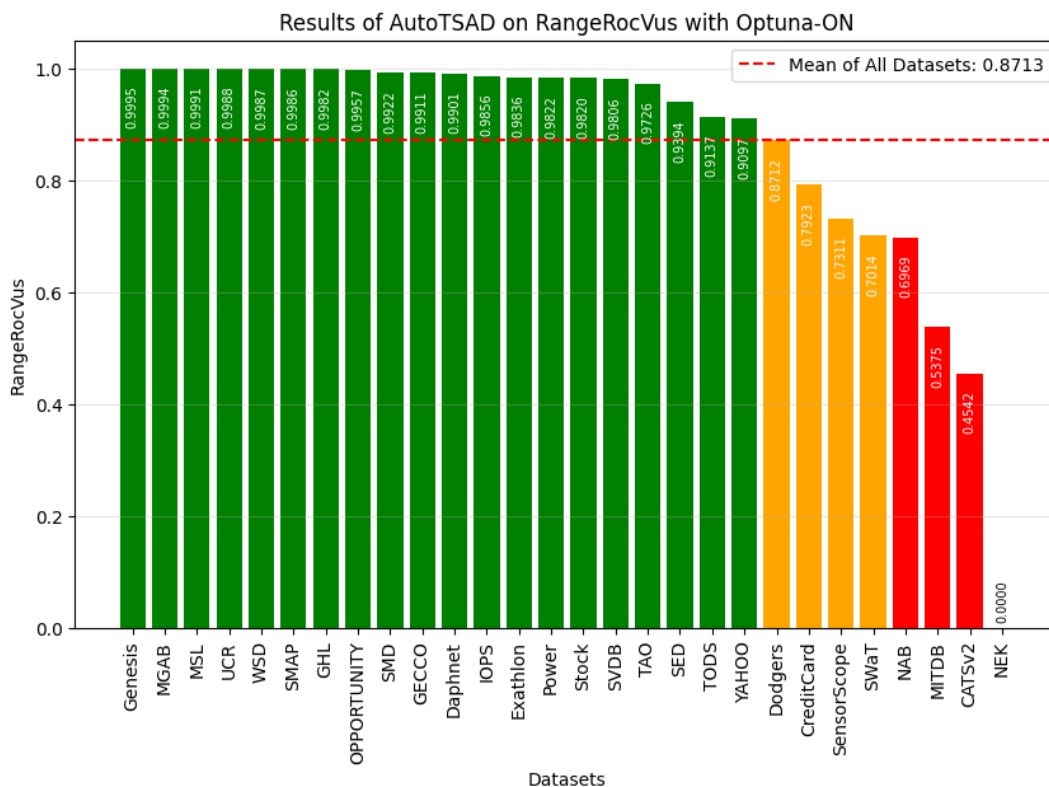
Έτσι θα έχουμε μία πιο σφαιρική άποψη για την επίδραση της Optuna στην εύρεση των ανωμαλιών.

Συγκριτικά αποτελέσματα με Optuna-ON

Τα αποτελέσματα της χρήσης της μηχανής της Optuna, είναι, όπως φαίνεται και στο Σχήμα 3.7, αρκετά εντυπωσιακά. Στα περισσότερα dataset, ανακάλυψε επιτυχημένα πάνω από το 90% των ανωμαλιών, ενώ σημαντικό είναι επίσης το γεγονός, πως σε αρκετά το ποσοστό επιτυχίας ήταν $\geq 99\%$. Ο μέσος όρος των αποτελεσμάτων ήταν 87,13%, ενώ στα dataset *NAB*, *MITDB*, *CATSv2*, δεν πέτυχε υψηλά ποσοστά επιτυχίας, καθώς επίσης και στο dataset *NEK* δεν μπόρεσε να βρεί καμία ανωμαλία, παρόλο που χρειάστηκε να δαπανηθούν 24 λεπτά για την εύρεση υπερπαραμέτρων. Από ότι φαίνεται από τα log files της εκτέλεσης, δεν μπόρεσε να βρεθεί κάποια περίοδος, τα δεδομένα εκπαίδευσης παράχτηκαν στα τυφλά, με αποτέλεσμα κατά την φάση της αποκοπής των αλγορίθμων, να επηζήσουν μόνο 2 διαφορετικές εκτελέσεις του αλγορίθμου *STOMP*, η μία μάλιστα με default παραμέτρους anomaly window size = 10, εκ των οποίων κανείς δεν μπορεί να παράξει αποτελέσματα.

Από τους αλγορίθμους που εκτελέστηκαν από το *TSB-UAD*, μπορούμε στο σχήμα 3.8 να παρατηρήσουμε την απόδοσή τους σε κάθε dataset μεμονωμένα, όπως επίσης και συγκριτικά πως τα πήγαν όλοι μαζί. Στο πάνω αριστερά σχήμα βλέπουμε την απόδοση του *Iforest*, ο οποίος φαίνεται ότι στα περισσότερα αποτελέσματα πέτυχε μία ικανοποιητική ανίχνευση ανωμαλιών, με χαμηλότερα αποτελέσματα να έχει πετύχει στα dataset *WSD* και *Power*. Αντίστοιχα πάνω δεξιά βλέπουμε την απόδοση του αλγορίθμου *POLY*, ο οποίος έχει εμφανώς χειρότερα αποτελέσματα από τον *Iforest*, και σε 2 dataset, στα *NEK* και *UCR* έχει πετύχει απόδοση μικρότερη του 20%. Έπειτα, στο κάτω αριστερά σχήμα βλέπουμε τα αποτελέσματα του αλγορίθμου *MP*, ο οποίος πάλι φαίνεται να έχει χειρότερα αποτελέσματα από τον *Iforest*, ενώ και αυτός έχει απόδοση μικρότερη του 20% σε 2 dataset, διαφορετικά από τον αλγόριθμο *POLY*, στα *IOPS* και *YAHOO*. Τέλος στο κάτω δεξιά σχήμα, μπορούμε να δούμε συγκεντρωτικά τις αποδόσεις μεταξύ των αλγορίθμων που έτρεξαν μέσω του *TSB-UAD*.

Ιδιαίτερο ενδιαφέρον έχει όταν συγκρίνουμε όλα τα αποτελέσματα μεταξύ τους, δηλαδή του *Auto-TSAD* και των αλγορίθμων του *TSB-UAD*. Συγκριτικά τα αποτελέσματα του νικητήριου αλγορίθμου ανά dataset φαίνονται στον Πίνακα 3.5. Όπως φαίνεται και στο 3.9, θα περίμενε κανείς το *Auto-TSAD* να τα έχει πάει καλύτερα σε όλα τα dataset. Κάτι τέτοιο όμως δεν είναι αληθές και βλέπουμε ότι έχει καταφέρει να φέρει καλύτερα αποτελέσματα, μόνο στο 57,16% των περιπτώσεων, γεγονός το οποίο είναι ιδιαίτερα χαμηλό αν αναλογιστεί κανείς το χρονικό κόστος. Η σημαντικότερη όμως παρατήρηση είναι, πως ακόμα και αν δεν κατάφερε να υπερτερήσει



Σχήμα 3.7: Συγκριτικά αποτελέσματα αλγορίθμων IForest - POLY - MP στην μετρική ROC-VUS.

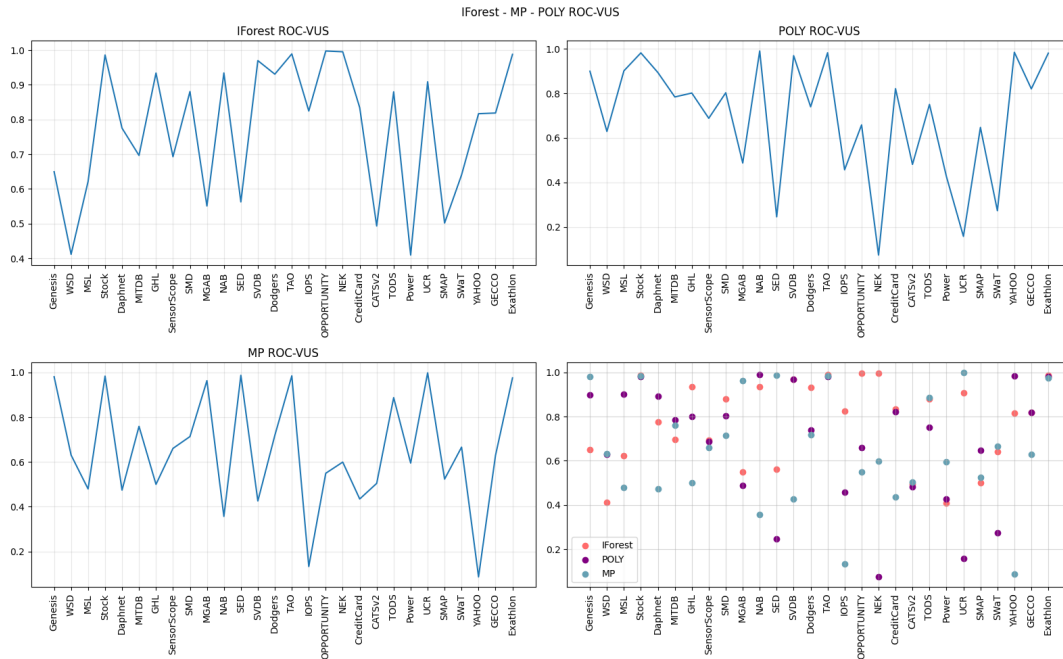
σε όλες τις περιπτώσεις, ήταν πολύ κοντά στην μέγιστη τιμή ανωμαλιών που ανιχνεύτηκε από τις άλλες τεχνικές.

Πιο συγκεκριμένα και όπως φαίνεται και στο boxplot του σχήματος 3.10, το *Auto-TSAD* έχει πολύ πιο σταθερή απόδοση σε σχέση με τους υπόλοιπους αλγορίθμους. Η διάμεσός του, η οποία δεν φαίνεται καθαρά, αγγίζει σχεδόν το 1.0 και το κουτί του, είναι εξαιρετικά συμπιεσμένο προς τα πάνω. Ακολουθεί ο αλγόριθμος *IForest* με καλύτερα αποτελέσματα από τους άλλους, ενώ ο *POLY* και ο *MP* έχουν σχεδόν παρόμοια συμπεριφορά. Το barchart του σχήματος 3.10 μας δείχνει ξεκάθαρα πως το *Auto-TSAD* κυριαρχεί έναντι των άλλων μεθόδων με 16 νίκες, έναντι του *IForest* με 8 νίκες και των *POLY* και *MP* με 3 και 2 νίκες αντιστοίχως. Στο 3ο διάγραμμα του σχήματος 3.10, όποια κουκίδα βρίσκεται κάτω και δεξιά από την γραμμή $y = x$, σημαίνει ότι το *Auto-TSAD* ξεπέρασε τον αντίστοιχο αλγόριθμο. Παρατηρήτε μία συγκέντρωση κουκκίδων, στην τέρμα δεξιά πλευρά, γεγονός το οποίο μας δείχνει πως όταν το *Auto-TSAD* κατάφερε να πιάσει σχεδόν 100% των ανωμαλιών, οι άλλοι αλγόριθμοι αποτύγχαναν αρκετές φορές.

Φτάνουμε λοιπόν στο συμπέρασμα πως υπάρχει ένα ξεκάθαρο αντιστάθισμα μεταξύ υπολογιστικού κόστους και ακρίβειας. Αν και η εκτέλεση μεμονωμένων αλγορίθμων απαιτεί μόνο μερικά δευτερόλεπτα με προκαθορισμένες υπερπαραμέτρους, η απόδοσή τους είναι εξαιρετικά ασταθής. Σε αντίθεση, οι τεχνικές βελτιστοποίησης του *Auto-TSAD* αν και εκτοξεύουν το χρόνο εκτέλεσης σε τάξης αρκετών ωρών ή ακόμα και

Πίνακας 3.5: Νικητήριοι Αλγόριθμοι ανά Dataset

	Winner	Score
Genesis	AutoTSAD	0.9995
WSD	AutoTSAD	0.9987
MSL	AutoTSAD	0.9991
Stock	IForest	0.9860
Daphnet	AutoTSAD	0.9901
MITDB	POLY	0.7839
GHL	AutoTSAD	0.9982
SensorScope	AutoTSAD	0.7311
SMD	AutoTSAD	0.9922
MGAB	AutoTSAD	0.9994
NAB	POLY	0.9902
SED	MP	0.9873
SVDB	AutoTSAD	0.9806
Dodgers	IForest	0.9307
TAO	IForest	0.9891
IOPS	AutoTSAD	0.9856
OPPORTUNITY	IForest	0.9975
NEK	IForest	0.9953
CreditCard	IForest	0.8342
CATSv2	MP	0.5044
TODS	AutoTSAD	0.9137
Power	AutoTSAD	0.9822
UCR	AutoTSAD	0.9988
SMAP	AutoTSAD	0.9986
SWaT	AutoTSAD	0.7014
YAHOO	POLY	0.9844
GECCO	AutoTSAD	0.9911
Exathlon	IForest	0.9879



Σχήμα 3.8: Αποτελέσματα χρήσης Optuna σε φθίνουσα σειρά.

ημερών, εξασφαλίζουν σταθερότητα και ανωτερότητα στην εύρεση ενωμάτων, εκτός μεμονωμένων περιπτώσεων.

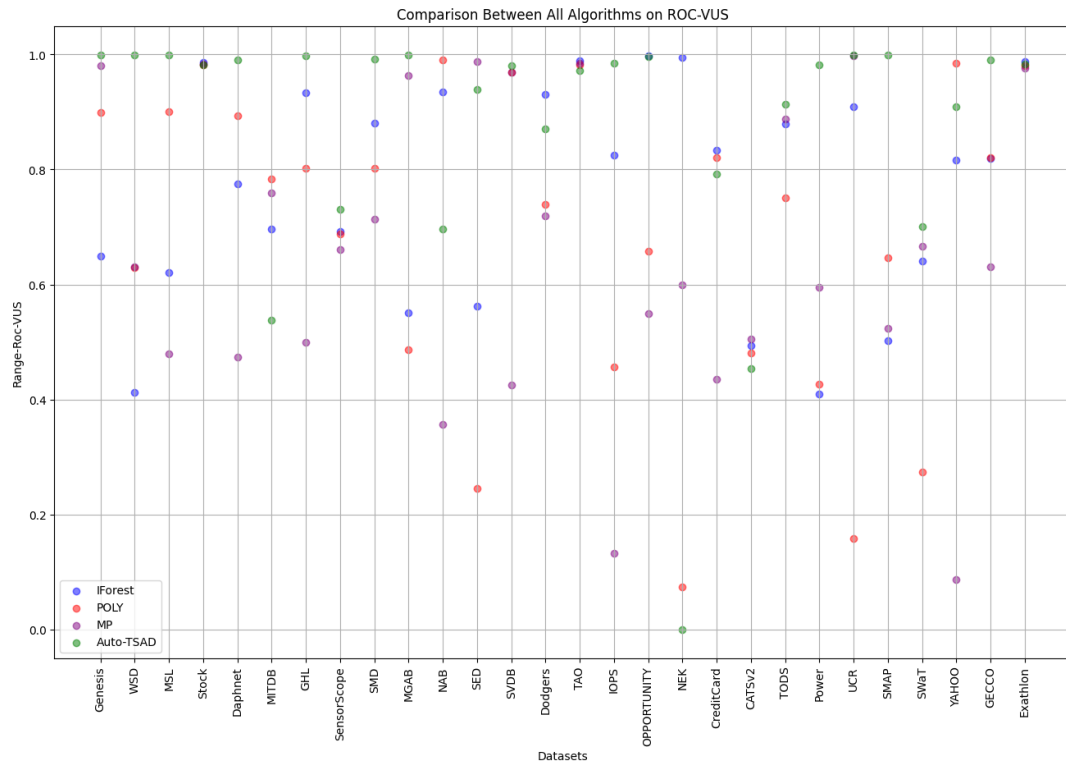
Συγκριτικά αποτελέσματα με Optuna-OFF

Όπως έχουμε αναφέρει και στην αρχή του κεφαλαίου, χωρίς την μηχανή βελτιστοποίησης *Optuna*, το *Auto-TSAD* μπόρεσε να υπολογίσει αποτελέσματα μόνο στο 31,2% των περιπτώσεων, ποσοστό αρκετά χαμηλό. Στο σχήμα 3.11 παρατηρούμε ότι οι τιμές της ROC-VUS διαφέρουν αρκετά σχετικά με το αν συμπεριλάβουμε τις αποτυχημένες προσπάθειες ή όχι. Συγκεκριμένα:

- **Με** συμπερίληψη των αποτυχημένων προσπαθειών, βλέπουμε ότι οι τιμές της ROC-VUS επηρεάζονται αρκετά από τις αποτυχίες και περιορίζονται στο διάστημα $[0, 0.4]$. Επίσης η διάμεσος είναι 0.33 και η τυπική απόκλιση 0.41.
- **Χωρίς** συμπερίληψη αποτυχημένων προσπαθειών, η εικόνα αλλάζει δραματικά, η ελάχιστη τιμή είναι περίπου 0.42, η μέγιστη αγγίζει το απόλυτο, ενώ υπάρχει μία συγκέντρωση τιμών στο διάστημα $[0.73, 0.96]$. Η διάμεσος είναι 0.84 ενώ η τυπική απόκλιση είναι 0.11.

Παρατηρούμε λοιπόν, πως υπάρχει σημαντική διαφορά των αποτελεσμάτων, αν δεν συμπεριλάβουμε τα αποτελέσματα των αποτυχημένων προσπαθειών.

Στο σχήμα 3.12, στο οποίο συγκρίνουμε την απόδοση του *Auto-TSAD* χωρίς *Optuna* με τους αλγορίθμους το *TSB-UAD*, βλέπουμε πως, όταν ο *Auto-TSAD* καταφέρνει να δώσει αποτέλεσμα, είναι αρκετά πιο αποτελεσματικούς από τους υπόλοιπους αλγορίθμους. Αν αντιθέτως συγκρίνουμε τα στατιστικά του συνυπολογίζοντας και τις αποτυχίες του, τότε



Σχήμα 3.9: Σύγκριση μεταξύ όλων των τεχνικών.

οι απλοί αλγόριθμοι υπερτερούν αρκετά. Το μεγάλο πλεονέκτημα των αλγορίθμων του *TSB-UAD*, είναι πως πάντα θα καταφέρουν να υπολογίσουν ένα μέρος των ανωμαλιών της χρονοσειράς που εξετάζουν, ακόμα και αν το αποτέλεσμα τους δεν είναι ικανοποιητικό.

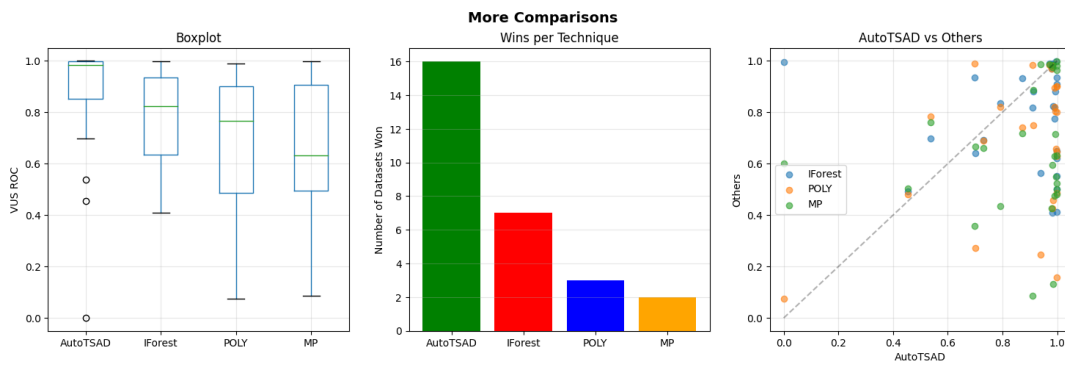
ΕΡΩΤΗΜΑ ΠΡΟΣ Κ.ΓΟΥΝΑΡΗ. ΑΝ ΘΕΛΕΤΕ ΜΠΟΡΩ ΚΑΙ ΕΔΩ ΝΑ ΚΑΝΩ ΑΝΑΛΥΣΗ ΑΝΑ DATASET, ΚΑΙ ΟΧΙ ΣΥΓΚΕΝΤΡΩΤΙΚΑ.

3.3 Artificial Dataset

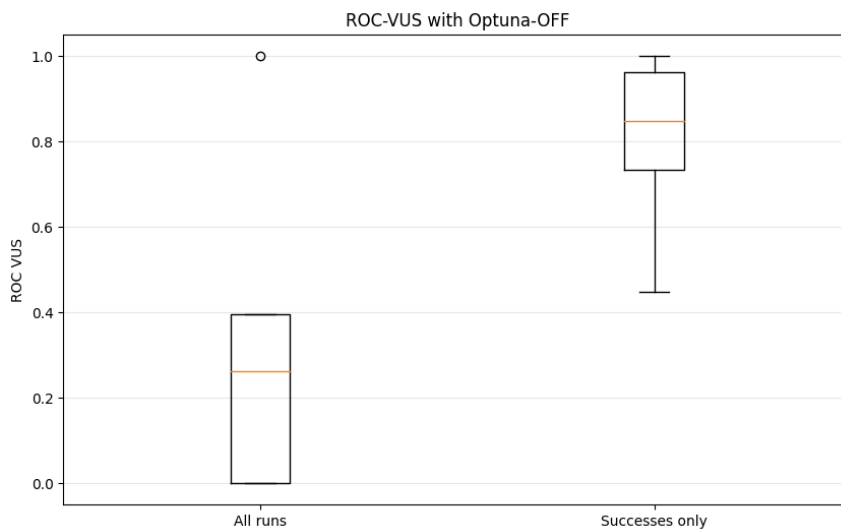
ΠΡΟΣ Κ.ΓΟΥΝΑΡΗ. ΕΔΩ ΘΑ ΕΧΩ ΤΗΝ ΙΔΙΑ ΑΝΑΛΥΣΗ ΟΠΩΣ ΣΤΟ PUBLIC DATASET, ΑΠΛΑ ΕΠΕΙΔΗ ΤΟ ARTIFICIAL ΔΕΝ ΧΩΡΙΖΕΙ ΣΕ ΔΙΑΦΟΡΕΤΙΚΟΥ ΤΥΠΟΥ ΔΕΔΟΜΕΝΩΝ Η ΑΝΑΛΥΣΗ ΘΑ ΕΙΝΑΙ ΠΙΟ ΣΥΝΤΟΜΗ

3.4 ΚΕΡΔΟΣ ΑΝΑ ΧΡΟΝΟ

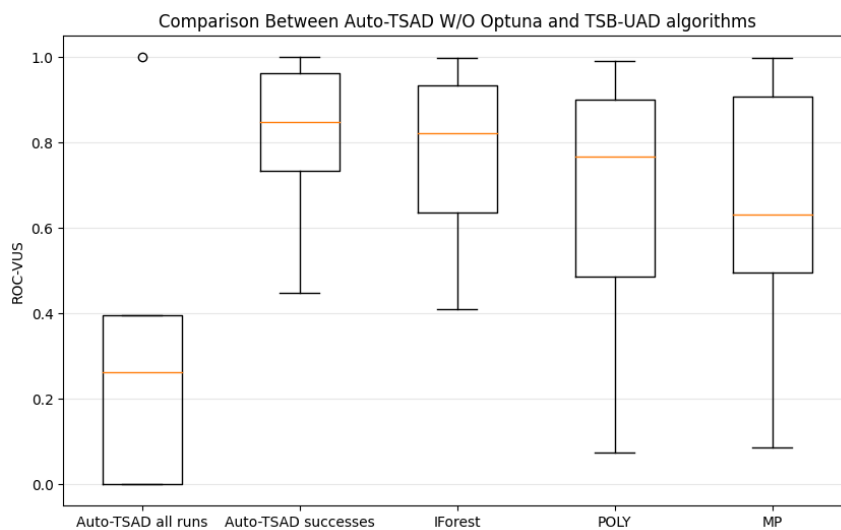
ΠΡΟΣ Κ.ΓΟΥΝΑΡΗ. ΕΔΩ ΘΑ ΒΑΛΩ ΑΥΤΟ ΠΟΥ ΜΕ ΕΙΠΑΤΕ, ΟΠΟΥ ΘΑ ΠΟΣΟ ΚΕΡΔΙΖΩ ΣΤΗΝ ROC VUS // ΜΟΝΑΔΕΣ ΧΡΟΝΟΥ



Σχήμα 3.10: Σύγκριση μεταξύ όλων των τεχνικών.



Σχήμα 3.11: Αποτελέσματα Auto-TSAD με Optuna-OFF στο v2_public dataset.



Σχήμα 3.12: Σύγκριση αποτελεσμάτων μεταξύ Auto-TSAD χωρίς Optuna και TSB-UAD.

Παράρτημα Α'

Frequently Asked Questions

A'.1 How do I change the colors of links?

The color of links can be changed to your liking using:

```
\hypersetup{urlcolor=red}, or  
\hypersetup{citecolor=green}, or  
\hypersetup{allcolor=blue}.
```

If you want to completely hide the links, you can use:

```
\hypersetup{allcolors=.}, or even better:  
\hypersetup{hidelinks}.
```

If you want to have obvious links in the PDF but not the printed text, use:

```
\hypersetup{colorlinks=false}.
```


Βιβλιογραφία

- [1] S. Schmidl, F. Naumann και T. Papenbrock, «AutoTSAD: unsupervised holistic anomaly detection for time series data», *Proceedings of the VLDB Endowment*, τόμ. 17, αρθμ. 11, σσ. 2987–3002, 2024.
- [2] J. Paparrizos, P. Boniol, T. Palpanas, R. S. Tsay, A. J. Elmore και M. J. Franklin, «Volume Under the Surface: A New Accuracy Evaluation Measure for Time-Series Anomaly Detection.», *Proc. VLDB Endow.*, τόμ. 15, αρθμ. 11, σσ. 2774–2787, 2022.
- [3] J. Paparrizos, Y. Kang, P. Boniol, R. S. Tsay, T. Palpanas και M. J. Franklin, «TSB-UAD: An End-to-End Benchmark Suite for Univariate Time-Series Anomaly Detection.», *Proc. VLDB Endow.*, τόμ. 15, αρθμ. 8, σσ. 1697–1711, 2022.
- [4] Y. Gao, J. Lin και C. Brif, «Ensemble grammar induction for detecting anomalies in time series», *arXiv preprint arXiv:2001.11102*, 2020.
- [5] F. T. Liu, K. M. Ting και Z.-H. Zhou, «Isolation forest», στο *2008 eighth ieee international conference on data mining*, IEEE, 2008, σσ. 413–422.
- [6] M. M. Breunig, H.-P. Kriegel, R. T. Ng και J. Sander, «LOF: identifying density-based local outliers», στο *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, 2000, σσ. 93–104.
- [7] P. Boniol, J. Paparrizos, T. Palpanas και M. J. Franklin, «SAND in Action: Subsequence Anomaly Detection for Streams.», *Proc. VLDB Endow.*, τόμ. 14, αρθμ. 12, σσ. 2867–2870, 2021.
- [8] P. Boniol, M. Linardi, F. Roncallo και T. Palpanas, «Automated anomaly detection in large sequences», στο *2020 IEEE 36th international conference on data engineering (ICDE)*, IEEE, 2020, σσ. 1834–1837.
- [9] P. Benecki, S. Piechaczek, D. Kostrzewa και J. Nalepa, «Detecting anomalies in spacecraft telemetry using evolutionary thresholding and LSTMs», στο *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2021, σσ. 143–144.
- [10] P. Boniol, M. Linardi, F. Roncallo, T. Palpanas, M. Meftah και E. Remy, «Unsupervised and scalable subsequence anomaly detection in large data series», *The VLDB Journal*, τόμ. 30, αρθμ. 6, σσ. 909–931, 2021.
- [11] P. Boniol, J. Paparrizos, Y. Kang κ.ά., «Theseus: Navigating the Labyrinth of Time-Series Anomaly Detection.», *Proc. VLDB Endow.*, τόμ. 15, αρθμ. 12, σσ. 3702–3705, 2022.

- [12] C.-C. M. Yeh, Y. Zhu, L. Ulanova κ.ά., «Matrix profile I: all pairs similarity joins for time series: a unifying view that includes motifs, discords and shapelets», στο *2016 IEEE 16th international conference on data mining (ICDM)*, Ieee, 2016, σσ. 1317–1322.
- [13] P. Boniol και T. Palpanas, «Series2graph: Graph-based subsequence anomaly detection for time series», *arXiv preprint arXiv:2207.12208*, 2022.
- [14] M. Linardi, Y. Zhu, T. Palpanas και E. Keogh, «Matrix profile goes MAD: variable-length motif and discord discovery in data series», *arXiv preprint arXiv:2008.13447*, 2020.
- [15] C.-C. M. Yeh, Y. Zhu, L. Ulanova κ.ά., «Time series joins, motifs, discords and shapelets: a unifying view that exploits the matrix profile», *Data Mining and Knowledge Discovery*, τόμ. 32, αρθμ. 1, σσ. 83–123, 2018.
- [16] P. Boniol, J. Paparrizos, T. Palpanas και M. J. Franklin, «SAND: Streaming Subsequence Anomaly Detection.», *Proc. VLDB Endow.*, τόμ. 14, αρθμ. 10, σσ. 1717–1729, 2021.
- [17] T. Akiba, S. Sano, T. Yanase, T. Ohta και M. Koyama, «Optuna: A next-generation hyperparameter optimization framework», στο *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, σσ. 2623–2631.
- [18] J. Li, A. Skjellum και R. D. Falgout, «A poly-algorithm for parallel dense matrix multiplication on two-dimensional process grid topologies», *Concurrency: Practice and Experience*, τόμ. 9, αρθμ. 5, σσ. 345–389, 1997.
- [19] P. Wenig, S. Schmidl και T. Papenbrock, «TimeEval: a benchmarking toolkit for time series anomaly detection algorithms», *Proc. VLDB Endow.*, τόμ. 15, αρθμ. 12, σσ. 3678–3681, Αύγ. 2022, issn: 2150-8097. doi: [10.14778/3554821.3554873](https://doi.org/10.14778/3554821.3554873). διεύθν.: <https://doi.org/10.14778/3554821.3554873>.
- [20] Z. Li, H. Ma και Y. Mei, «A unifying method for outlier and change detection from data streams based on local polynomial fitting», στο *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, Springer, 2007, σσ. 150–161.